



Εθνικό και Καποδιστριακό Πανεπιστήμιο Αθηνών
Τμήμα Πληροφορικής και Τηλεπικοινωνιών

Πτυχιακή Εργασία
Σειριακοί και Παράλληλοι Αλγόριθμοι

Καραγιάννης Δημήτριος
1115202200293
sdi2200293@di.uoa.gr

Επιβλέπων Καθηγητής: ΕΔΙΠ Δρ. Λουκά Μαρία

Καλοκαίρι 2026

Περιεχόμενα

Εισαγωγή	3
1 Αλγόριθμοι Γραφημάτων	4
1.1 Ορισμοί και Αναπαράσταση	4
2 Ελάχιστο δέντρο επικάλυψης: Ο αλγόριθμος του Prim	7
2.1 Παράλληλη Υλοποίηση	10
3 Συντομότερες Διαδρομές από Μία Πηγή: Ο αλγόριθμος του Dijkstra	13
3.1 Παράλληλη Υλοποίηση	14
3.1.1 API/Σύστημα μεταβίβασης μηνυμάτων	15
3.1.2 API/Σύστημα κοινής διεύθυνσης	15
3.1.3 Γλώσσα	15
3.1.4 Περιβάλλον συστήματος	15
3.2 Δεδομένα δοκιμών	15
3.3 Αλγόριθμος	16
3.3.1 Υλοποίηση του αλγορίθμου του Dijkstra με MPI	16
3.3.2 Υλοποίηση του αλγορίθμου του Dijkstra με OpenMP	18
3.4 Αποτελέσματα Υλοποίησης και Ανάλυση	19
3.5 Συμπεράσματα και Περαιτέρω Έρευνα	25
4 Συντομότερες Διαδρομές για Όλα τα Ζεύγη	26
4.1 Αλγόριθμος βασισμένος στον Πολλαπλασιασμό Πίνακα	26
4.2 Ο αλγόριθμος του Dijkstra	28
4.3 Ο αλγόριθμος του Floyd	31
4.4 Συγκρίσεις Απόδοσης	37
5 Μεταβατική Κλειστότητα	38
6 Συνεκτικές Συνιστώσες	38
6.1 Αλγόριθμος Βασισμένος στην Αναζήτηση Πρώτα Κατά Βάθος	39
7 Αλγόριθμοι για Αραιά Γραφήματα	42
7.1 Συντομότερα Μονοπάτια Μονής Πηγής	45
Βιβλιογραφία	49

Εισαγωγή

Σύντομη εισαγωγή...

1 Αλγόριθμοι Γραφημάτων

Η θεωρία γραφημάτων διαδραματίζει σημαντικό ρόλο στην επιστήμη των υπολογιστών, επειδή παρέχει έναν εύκολο και συστηματικό τρόπο για τη μοντελοποίηση πολλών προβλημάτων. Πολλά προβλήματα μπορούν να εκφραστούν με όρους γραφημάτων και να επιλυθούν με τη χρήση τυπικών αλγορίθμων γραφημάτων. Το παρόν κεφάλαιο παρουσιάζει παράλληλες υλοποιήσεις κάποιων σημαντικών και θεμελιωδών αλγορίθμων γραφημάτων.

1.1 Ορισμοί και Αναπαράσταση

Ένα μη κατευθυνόμενο γράφημα G είναι ένα ζεύγος (V, E) , όπου V είναι ένα πεπερασμένο σύνολο σημείων που ονομάζονται κορυφές και E είναι ένα πεπερασμένο σύνολο ακμών. Μια ακμή $e \in E$ είναι ένα μη διατεταγμένο ζεύγος (u, v) , όπου $(u, v) \in V$. Μια ακμή (u, v) υποδηλώνει ότι οι κορυφές u και v είναι συνδεδεμένες. Ομοίως, ένα κατευθυνόμενο γράφημα G είναι ένα ζεύγος (V, E) , όπου V είναι το σύνολο των κορυφών όπως μόλις ορίσαμε, αλλά μια ακμή $(u, v) \in E$ είναι ένα διατεταγμένο ζεύγος, δηλαδή υποδηλώνει ότι υπάρχει σύνδεση από το u στο v . Το Σχήμα 1 απεικονίζει ένα μη κατευθυνόμενο και ένα κατευθυνόμενο γράφημα. Χρησιμοποιούμε τον όρο γράφημα για να αναφερθούμε τόσο στα κατευθυνόμενα όσο και στα μη κατευθυνόμενα γραφήματα.

Πολλοί ορισμοί είναι κοινοί για κατευθυνόμενα και μη κατευθυνόμενα γραφήματα, αν και ορισμένοι όροι έχουν ελαφρώς διαφορετική σημασία στο καθένα. Αν η (u, v) είναι μια ακμή σε ένα μη κατευθυνόμενο γράφημα, τότε η (u, v) συνδέει (**incident on**) τις κορυφές u και v . Ωστόσο, αν το γράφημα είναι κατευθυνόμενο, τότε η ακμή (u, v) ξεκινάει (**incident from**) από την κορυφή u και καταλήγει (**incident to**) στην κορυφή v . Για παράδειγμα, στο Σχήμα 1(α), η ακμή e συνδέει (**incident on**) τις κορυφές 5 και 4, ενώ στο Σχήμα 1(β), η ακμή f ξεκινάει (**incident from**) από την κορυφή 5 και καταλήγει (**incident to**) στην κορυφή 2. Αν η (u, v) είναι μια ακμή σε έναν μη κατευθυνόμενο γράφημα $G = (V, E)$, οι κορυφές u και v λέγεται ότι είναι γειτονικές (**adjacent**) μεταξύ τους. Αν το γράφημα είναι κατευθυνόμενο, η κορυφή v λέγεται ότι είναι γειτονική της (**adjacent to**) κορυφής u .



Σχήμα 1: (α) Ένα μη κατευθυνόμενο γράφημα και (β) ένα κατευθυνόμενο γράφημα.

Μια διαδρομή (ή μονοπάτι) από μια κορυφή v σε μια κορυφή u είναι μια ακολουθία $< v_0, v_1, v_2, \dots, v_k >$ κορυφών, όπου $v_0 = v$, $v_k = u$, και $(v_i, v_{i+1}) \in E$ για $i = 0, 1, \dots, k-1$.



$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 1 & 0 \end{bmatrix}$$

Σχήμα 2: Ένα μη κατευθυνόμενο γράφημα και η αναπαράστασή του με λίστα γειτνίασης.

Το μήκος μιας διαδρομής ορίζεται ως το πλήθος των ακμών σε αυτή τη διαδρομή. Αν υπάρχει διαδρομή από το v στο u , τότε το u είναι προσπελάσιμο (**reachable**) από το v . Μια διαδρομή είναι απλή αν όλες οι κορυφές της είναι διακριτές (δεν επαναλαμβάνονται). Μια διαδρομή σχηματίζει κύκλο αν η αρχική και η τελική της κορυφή ταυτίζονται – δηλαδή, αν $v_0 = v_k$. Ένα γράφημα χωρίς κύκλους ονομάζεται ακυκλικό. Ένας κύκλος είναι απλός αν όλες οι ενδιαμέσες κορυφές του είναι διακριτές.

Για παράδειγμα, στο Σχήμα 1(α), η ακολουθία $\langle 3, 6, 5, 4 \rangle$ είναι μια διαδρομή από την κορυφή 3 στην κορυφή 4. Στο Σχήμα 1(β), υπάρχει ένας κατευθυνόμενος απλός κύκλος $\langle 1, 5, 6, 4, 1 \rangle$. Επίσης, στο Σχήμα 1(α), η ακολουθία $\langle 1, 2, 5, 3, 6, 5, 4, 1 \rangle$ είναι ένας μη κατευθυνόμενος κύκλος που δεν είναι απλός, γιατί περιέχει τον βρόχο $\langle 5, 3, 6, 5 \rangle$.

Ένα μη κατευθυνόμενο γράφημα είναι συνεκτικό αν κάθε ζεύγος κορυφών ενώνεται με μια διαδρομή. Λέμε ότι ένα γράφημα $G' = (V', E')$ είναι υπογράφημα του $G = (V, E)$ αν $V' \subseteq V$ και $E' \subseteq E$. Για ένα δεδομένο σύνολο $V' \subseteq V$, το υπόγραφο του G που προκύπτει από το V' είναι το γράφημα $G' = (V', E')$, όπου $E' = \{(u, v) \in E \mid u, v \in V'\}$. Ένα πλήρες γράφημα είναι ένα γράφημα στον οποίο κάθε ζεύγος κορυφών είναι γειτονικό. Ένα δάσος είναι ένα ακυκλικό γράφημα και ένα δέντρο είναι ένα συνεκτικό ακυκλικό γράφημα. Σημειώστε ότι αν $G = (V, E)$ είναι ένα δέντρο, τότε $|E| = |V| - 1$.

Μερικές φορές τα βάρη συσχετίζονται με κάθε ακμή στο E . Τα βάρη είναι συνήθως πραγματικοί αριθμοί που αντιπροσωπεύουν το κόστος ή το όφελος της διαδρομής κατά μήκος της συνδεδεμένης ακμής. Για παράδειγμα, σε ένα ηλεκτρονικό κύκλωμα, μια αντίσταση μπορεί να αναπαρασταθεί από μια ακμή της οποίας το βάρος είναι η τιμή της αντίστασής της. Ένα γράφημα που έχει βάρη συσχετισμένα με κάθε ακμή ονομάζεται γράφημα με βάρη και συμβολίζεται ως $G = (V, E, w)$, όπου τα V και E είναι όπως ορίσαμε προηγουμένως και η $w : E \rightarrow \mathbb{R}$ είναι μια πραγματική συνάρτηση που ορίζεται στο E . Το βάρος ενός γραφήματος ορίζεται ως το άθροισμα των βαρών των ακμών του. Το βάρος μιας διαδρομής είναι το άθροισμα των βαρών των ακμών της.

Υπάρχουν δύο τυπικές μέθοδοι για την αναπαράσταση ενός γραφήματος σε ένα πρόγραμμα υπολογιστή. Η πρώτη μέθοδος είναι η χρήση πίνακα (μήτρας), και η δεύτερη μέθοδος είναι η χρήση συνδεδεμένης λίστας.

Θεωρήστε ένα γράφημα $G = (V, E)$ με n κορυφές αριθμημένες ως $1, 2, \dots, n$. Ο πίνακας γειτνίασης αυτού του γραφήματος είναι ένας $n \times n$ πίνακας $A = (a_{i,j})$, ο οποίος ορίζεται ως εξής:

$$a_{i,j} = \begin{cases} 1 & \text{εάν } (v_i, v_j) \in E \\ 0 & \text{διαφορετικά} \end{cases}$$

Το Σχήμα 2 απεικονίζει μια αναπαράσταση με πίνακα γειτνίασης για έναν μη κατευθυνόμενο γράφημα. Σημειώστε ότι ο πίνακας γειτνίασης ενός μη κατευθυνόμενου γραφήματος είναι συμμετρικός. Η αναπαράσταση με πίνακα γειτνίασης μπορεί να τροποποιηθεί για να υποστηρίξει γραφήματα με βάρη. Σε αυτή την περίπτωση, ο $A = (a_{i,j})$ ορίζεται ως εξής:

$$a_{i,j} = \begin{cases} w(v_i, v_j) & \text{εάν } (v_i, v_j) \in E \\ 0 & \text{εάν } i = j \\ \infty & \text{διαφορετικά} \end{cases}$$

Αναφερόμαστε σε αυτόν τον τροποποιημένο πίνακα γειτνίασης ως πίνακα γειτνίασης με βάρη. Ο χώρος που απαιτείται για την αποθήκευση του πίνακα γειτνίασης ενός γραφήματος με n κορυφές είναι $\Theta(n^2)$.

Η αναπαράσταση με λίστα γειτνίασης ενός γραφήματος $G = (V, E)$ αποτελείται από έναν πίνακα $Adj[1..|V|]$ λιστών. Για κάθε $v \in V$, η $Adj[v]$ είναι μια συνδεδεμένη λίστα όλων των κορυφών u για τις οποίες ο G περιέχει μια ακμή $(v, u) \in E$. Με άλλα λόγια, η $Adj[v]$ είναι μια λίστα όλων των κορυφών που είναι γειτονικές προς την v . Το Σχήμα 3 δείχνει ένα παράδειγμα αναπαράστασης με λίστα γειτνίασης. Η αναπαράσταση με λίστα γειτνίασης μπορεί να τροποποιηθεί για να φιλοξενήσει γραφήματα με βάρη, αποθηκεύοντας το βάρος κάθε ακμής $(v, u) \in E$ στη λίστα γειτνίασης της κορυφής v . Ο χώρος που απαιτείται για την αποθήκευση της λίστας γειτνίασης είναι $\Theta(|E|)$.



Σχήμα 3: Ένα μη κατευθυνόμενο γράφημα και η αναπαράσταση του με λίστα γειτνίασης.

Η φύση του γραφήματος καθορίζει ποια αναπαράσταση πρέπει να χρησιμοποιηθεί. Ένα γράφημα $G = (V, E)$ είναι αραιό αν το $|E|$ είναι πολύ μικρότερο από $O(|V|^2)$, διαφορετικά είναι πυκνό. Η αναπαράσταση με πίνακα γειτνίασης είναι χρήσιμη για πυκνά γραφήματα, ενώ η αναπαράσταση με λίστα γειτνίασης είναι συχνά πιο αποδοτική για αραιά γραφήματα. Να σημειωθεί ότι ο ακολουθιακός χρόνος εκτέλεσης ενός αλγορίθμου που χρησιμοποιεί πίνακα γειτνίασης και χρειάζεται να διατρέξει όλες τις ακμές του γραφήματος είναι κάτω φραγμένος από $\Omega(|V|^2)$, επειδή πρέπει να προσπελαστεί ολόκληρος ο πίνακας. Ωστόσο, αν χρησιμοποιείται αναπαράσταση με λίστα γειτνίασης, ο χρόνος εκτέλεσης είναι κάτω φραγμένος από

$\Omega(|V| + |E|)$ για τον ίδιο λόγο. Επομένως, αν το γράφημα είναι αραιό (το $|E|$ είναι πολύ μικρότερο από $|V|^2$), η αναπαράσταση με λίστα γειτνίασης είναι καλύτερη από την αναπαράσταση με πίνακα γειτνίασης.

Το υπόλοιπο αυτού του κεφαλαίου παρουσιάζει διάφορους αλγορίθμους γραφημάτων. Τα πρώτα τέσσερα εδάφια παρουσιάζουν αλγορίθμους για πυκνά γραφήματα, και το τελευταίο εδάφιο συζητά αλγορίθμους για αραιά γραφήματα. Υποθέτουμε ότι τα πυκνά γραφήματα αναπαρίστανται με έναν πίνακα γειτνίασης και αραιά γραφήματα με μια λίστα γειτνίασης. Σε όλο αυτό το κεφάλαιο, το n συμβολίζει τον αριθμό των κορυφών στο γράφημα.

2 Ελάχιστο δέντρο επικάλυψης: Ο αλγόριθμος του Prim



Σχήμα 4: Ένα μη κατευθυνόμενο γράφημα και το ελάχιστο δέντρο επικάλυψης του.

Ένα δέντρο επικάλυψης ενός μη κατευθυνόμενου γραφήματος G είναι ένα υπογράφημα του G που είναι ένα δέντρο που περιέχει όλες τις κορυφές του G . Σε ένα γράφημα με βάρη, το βάρος ενός υπογραφήματος είναι το άθροισμα των βαρών των ακμών στο υπογράφημα. Ένα ελάχιστο δέντρο επικάλυψης (MST) για ένα μη κατευθυνόμενο γράφημα με βάρη είναι ένα δέντρο επικάλυψης με ελάχιστο βάρος. Πολλά προβλήματα απαιτούν την εύρεση του MST ενός μη κατευθυνόμενου γραφήματος. Για παράδειγμα, το ελάχιστο μήκος καλωδίου που απαιτείται για τη σύνδεση ενός συνόλου υπολογιστών σε ένα δίκτυο μπορεί να προσδιοριστεί με την εύρεση του MST του μη κατευθυνόμενου γραφήματος που περιέχει όλες τις πιθανές συνδέσεις. Το Σχήμα 4 απεικονίζει ένα MST ενός μη κατευθυνόμενου γραφήματος.

Αν το G δεν είναι συνεκτικό, δεν μπορεί να έχει δέντρο επικάλυψης. Αντ' αυτού, έχει δάσος επικάλυψης. Για λόγους απλότητας στην περιγραφή του αλγορίθμου ελάχιστου δέντρου επικάλυψης, υποθέτουμε ότι το G είναι συνεκτικό. Αν το G δεν είναι συνεκτικό, μπορούμε να βρούμε τις συνεκτικές συνιστώσες του (Ενότητα 6) και να εφαρμόσουμε τον αλγόριθμο ελάχιστου δέντρου επικάλυψης σε καθεμία από αυτές. Εναλλακτικά, μπορούμε να τροποποιήσουμε τον αλγόριθμο ελάχιστου δέντρου επικάλυψης έτσι ώστε να παράγει ένα ελάχιστο δάσος επικάλυψης.

```

1: procedure PRIM_MST( $V, E, w, r$ )
2:    $V_T \leftarrow \{r\}$ 
3:    $d[r] \leftarrow 0$ 
4:   for all  $v \in (V - V_T)$  do
5:     if edge  $(r, v)$  exists then
6:        $d[v] \leftarrow w(r, v)$ 
7:     else
8:        $d[v] \leftarrow \infty$ 
9:     end if
10:  end for
11:  while  $V_T \neq V$  do
12:    find vertex  $u$  such that  $d[u] = \min\{d[v] \mid v \in (V - V_T)\}$ 
13:     $V_T \leftarrow V_T \cup \{u\}$ 
14:    for all  $v \in (V - V_T)$  do
15:       $d[v] \leftarrow \min\{d[v], w(u, v)\}$ 
16:    end for
17:  end while
18: end procedure

```

Αλγόριθμος 1: Ο σειριακός αλγόριθμος του Prim για το ελάχιστο δέντρο επικάλυψης.

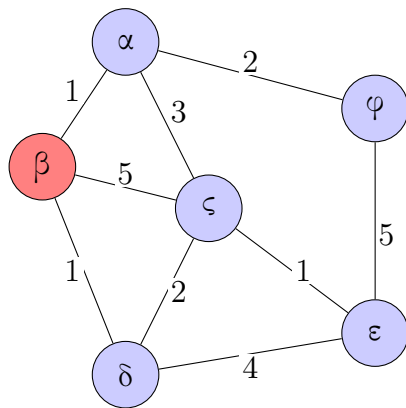
Ο αλγόριθμος του Prim για την εύρεση ενός MST είναι ένας αλγόριθμος απληστίας. Ο αλγόριθμος ξεκινά με την επιλογή μιας τυχαίας αρχικής κορυφής. Στη συνέχεια, αναπτύσσει το ελάχιστο δέντρο επικάλυψης επιλέγοντας μια νέα κορυφή και ακμή που είναι εγγυημένα ότι βρίσκονται σε ένα δέντρο επικάλυψης ελάχιστου κόστους. Ο αλγόριθμος συνεχίζει μέχρι να επιλεγούν όλες οι κορυφές.

Έστω $G = (V, E, w)$ το μη κατευθυνόμενο γράφημα με βάρη για το οποίο πρέπει να βρεθεί το ελάχιστο δέντρο επικάλυψης και έστω $A = (a_{i,j})$ ο πίνακας γειτνίασης με βάρη. Ο αλγόριθμος του Prim απεικονίζεται στον Αλγόριθμο 1. Ο αλγόριθμος χρησιμοποιεί το σύνολο V_T για να διατηρήσει τις κορυφές του ελάχιστου δέντρου επικάλυψης κατά τη διάρκεια της κατασκευής του. Χρησιμοποιεί επίσης έναν πίνακα $d[1..n]$ στον οποίο, για κάθε κορυφή $v \in (V - V_T)$, το $d[v]$ περιέχει το βάρος της ακμής με το ελάχιστο βάρος από οποιαδήποτε κορυφή στο V_T προς την κορυφή v . Αρχικά, το V_T περιέχει μια τυχαία κορυφή r που γίνεται η ρίζα του MST. Επιπλέον, $d[r] = 0$, και για όλα τα v τέτοια ώστε $v \in (V - V_T)$, $d[v] = w(r, v)$ αν υπάρχει τέτοια ακμή, διαφορετικά $d[v] = \infty$. Κατά τη διάρκεια κάθε επανάληψης του αλγορίθμου, μια νέα κορυφή u προστίθεται στο V_T έτσι ώστε $d[u] = \min\{d[v] \mid v \in (V - V_T)\}$. Μετά την προσθήκη αυτής της κορυφής, όλες οι τιμές του $d[v]$ έτσι ώστε $v \in (V - V_T)$ ενημερώνονται, επειδή τώρα μπορεί να υπάρχει μια ακμή με μικρότερο βάρος μεταξύ της κορυφής v και της νέας κορυφής u . Ο αλγόριθμος τερματίζεται όταν $V_T = V$. Το Σχήμα 5 απεικονίζει τον αλγόριθμο. Με την ολοκλήρωση του αλγορίθμου Prim, το κόστος του ελάχιστου δέντρου επικάλυψης είναι $\sum_{v \in V} d[v]$. Ο Αλγόριθμος 1 μπορεί εύκολα να τροποποιηθεί για να αποθηκεύει τις ακμές που ανήκουν στο ελάχιστο δέντρο επικάλυψης.

Στον Αλγόριθμο 1, το σώμα του βρόχου *while* (γραμμές 10-13) εκτελείται $n - 1$ φορές. Τόσο ο υπολογισμός του $\min\{d[v] \mid v \in (V - V_T)\}$ (γραμμή 10), όσο και ο βρόχος *for* (γραμμές 12 και 13) εκτελούνται σε $O(n)$ βήματα. Έτσι, η συνολική πολυπλοκότητα του αλγορίθμου του Prim είναι $\Theta(n^2)$.

Παρακάτω παρουσιάζεται ο αλγόριθμος του Prim. Το MST ξεκινάει από την κορυφή b . Κατά την διάρκεια κάθε επανάληψης, η ακμή με το ελάχιστο κόστος που συνδέει μια κορυφή του V_T με μια κορυφή του $V - V_T$, επιλέγεται και η αντίστοιχη κορυφή προστίθεται στο V_T , (όπως φαίνεται με **έντονη** γραμμή στον πίνακα d). Κατόπιν, ενημερώνονται οι τιμές του πίνακα $d[v]$ των κορυφών του $V - V_T$.

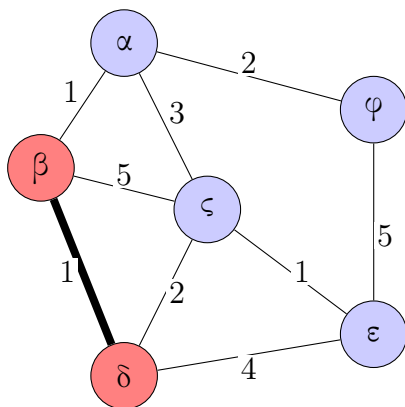
(α) Αρχικό γράφημα



	a	b	c	d	e	f
$d[]$	1	0	5	1	∞	∞

	a	b	c	d	e	f
a	0	1	3	∞	∞	2
b	1	0	5	1	∞	∞
c	3	5	0	2	1	∞
d	∞	1	2	0	4	∞
e	∞	∞	1	4	0	5
f	2	∞	∞	∞	5	0

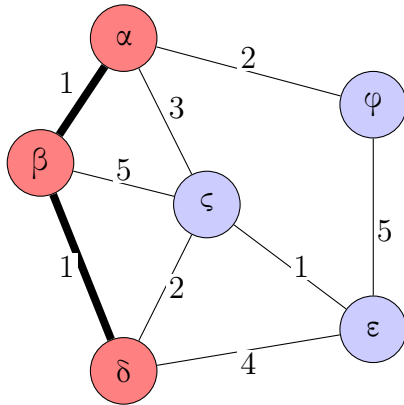
(β) Μετά την επιλογή της πρώτης ακμής



	a	b	c	d	e	f
$d[]$	1	0	2	1	4	∞

	a	b	c	d	e	f
a	0	1	3	∞	∞	2
b	1	0	5	1	∞	∞
c	3	5	0	2	1	∞
d	∞	1	2	0	4	∞
e	∞	∞	1	4	0	5
f	2	∞	∞	∞	5	0

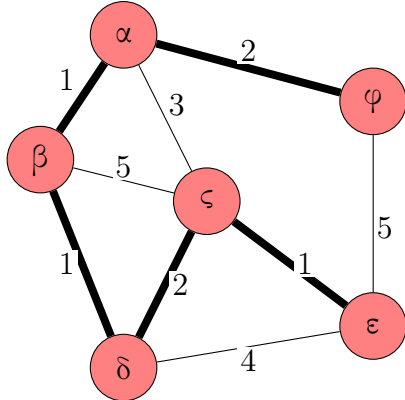
(γ) Μετά την επιλογή της δεύτερης ακμής



	a	b	c	d	e	f
$d[]$	1	0	2	1	4	2

	a	b	c	d	e	f
a	0	1	3	∞	∞	2
b	1	0	5	1	∞	∞
c	3	5	0	2	1	∞
d	∞	1	2	0	4	∞
e	∞	∞	1	4	0	5
f	2	∞	∞	∞	5	0

(δ) Τελικό ελάχιστο δέντρο επικάλυψης



	a	b	c	d	e	f
$d[]$	1	0	2	1	1	2

	a	b	c	d	e	f
a	0	1	3	∞	∞	2
b	1	0	5	1	∞	∞
c	3	5	0	2	1	∞
d	∞	1	2	0	4	∞
e	∞	∞	1	4	0	5
f	2	∞	∞	∞	5	0

Σχήμα 5: Ο αλγόριθμος του Prim για το ελάχιστο δέντρο επικάλυψης. Το MST έχει ρίζα την κορυφή β. Κατά τη διάρκεια κάθε επανάληψης, επιλέγεται η ακμή ελάχιστου κόστους που συνδέει μια κορυφή στο V_T με μια κορυφή στο $V - V_T$ και η αντίστοιχη κορυφή προστίθεται στο V_T (φαίνεται σκιασμένη στον πίνακα απόστασης d). Στη συνέχεια, οι τιμές $d[v]$ των κορυφών στο $V - V_T$ ενημερώνονται.

2.1 Παράλληλη Υλοποίηση

Ο αλγόριθμος του Prim είναι επαναληπτικός. Κάθε επανάληψη προσθέτει μια νέα κορυφή στο ελάχιστο δέντρο επικάλυψης. Δεδομένου ότι η τιμή του $d[v]$ για μια κορυφή v μπορεί να



Σχήμα 6: Η διαμέριση του πίνακα απόστασης d και ο πίνακας γειτνίασης A μεταξύ p επεξεργαστών.

αλλάζει κάθε φορά που προστίθεται μια νέα κορυφή u στο V_T , είναι δύσκολο να επιλεγούν περισσότερες από μία κορυφές για να συμπεριληφθούν στο ελάχιστο δέντρο επικάλυψης. Για παράδειγμα, στο γράφημα του Σχήματος 5, έπειτα από την επιλογή της κορυφής b , εάν και οι δύο κορυφές d και c επιλεγούν, το MST δεν θα βρεθεί. Αυτό συμβαίνει γιατί, έπειτα από την επιλογή της κορυφής d , η τιμή της $d[c]$ ενημερώνεται από 5 σε 2. Επομένως, δεν είναι εύκολο να εκτελεστούν παράλληλα διαφορετικές επαναλήψεις του βρόχου *while*. Ωστόσο, κάθε επανάληψη μπορεί να εκτελεστεί παράλληλα ως εξής.

Έστω p ο αριθμός των επεξεργαστών και n ο αριθμός των κορυφών στο γράφημα. Το σύνολο V χωρίζεται σε p υποσύνολα χρησιμοποιώντας την *block* διαμέριση σε λωρίδες. Κάθε υποσύνολο έχει $\frac{n}{p}$ συνεχόμενες κορυφές και η εργασία που σχετίζεται με κάθε υποσύνολο ανατίθεται σε διαφορετικό επεξεργαστή. Έστω V_i το υποσύνολο των κορυφών που έχουν ανατεθεί στον επεξεργαστή P_i για $i = 0, 1, \dots, p - 1$. Κάθε επεξεργαστής P_i αποθηκεύει το τμήμα του πίνακα d που αντιστοιχεί στο V_i (δηλαδή, ο επεξεργαστής P_i αποθηκεύει το $d[v]$ έτσι ώστε $v \in V_i$). Το Σχήμα 6(α) απεικονίζει την διαμέριση. Κάθε επεξεργαστής P_i υπολογίζει $d_i[u] = \min\{d_i[v] \mid v \in (V - V_T) \cap V_i\}$ κατά τη διάρκεια κάθε επανάληψης του βρόχου *while*. Το συνολικό ελάχιστο λαμβάνεται τότε για όλα τα $d_i[u]$ χρησιμοποιώντας την ένα-προς-όλους μετάδοση και αποθηκεύεται στον επεξεργαστή P_0 . Ο επεξεργαστής P_0 διατηρεί τώρα τη νέα κορυφή u , η οποία θα εισαχθεί στο V_T . Ο επεξεργαστής P_0 μεταδίδει το u σε όλους τους επεξεργαστές χρησιμοποιώντας την ένα-προς-όλους μετάδοση. Ο επεξεργαστής P_i που είναι υπεύθυνος για την κορυφή u σημειώνει το u ότι ανήκει στο σύνολο V_T . Τέλος, κάθε επεξεργαστής ενημερώνει τις τιμές του $d[v]$ για τις τοπικές κορυφές του.

Όταν μια νέα κορυφή u εισέρχεται στο V_T , οι τιμές του $d[v]$ για το $v \in (V - V_T)$ πρέπει να ενημερωθούν. Ο επεξεργαστής που είναι υπεύθυνος για το v πρέπει να γνωρίζει το βάρος της ακμής (u, v) . Επομένως, κάθε επεξεργαστής P_i πρέπει να αποθηκεύει τις στήλες του πίνακα γειτνίασης με βάρη που αντιστοιχούν στο σύνολο V_i των κορυφών που έχουν εκχωρηθεί σε αυτόν. Ο χώρος για την αποθήκευση του απαιτούμενου τμήματος του πίνακα γειτνίασης

με βάρη σε κάθε επεξεργαστή είναι $\Theta\left(\frac{n^2}{p}\right)$. Το Σχήμα 6(β) απεικονίζει τον διαμερισμό του πίνακα γειτνίασης με βάρη. Ο υπολογισμός που εκτελείται από έναν επεξεργαστή για την ελαχιστοποίηση και την ενημέρωση των τιμών του $d[v]$ κατά τη διάρκεια κάθε επανάληψης είναι $\Theta\left(\frac{n}{p}\right)$. Η επικοινωνία που εκτελείται σε κάθε επανάληψη οφείλεται στη μονού κόμβου συσσώρευση και στη μετάδοση ενός προς όλους.

Υπερκύβος

Για έναν υπερκύβο με p -επεξεργαστές, η μια-προς-όλους μετάδοση μιας λέξης από έναν προς όλους απαιτεί χρόνο $(t_s + t_w) \log p$. Η εύρεση του συνολικού ελάχιστου μιας λέξης σε κάθε επεξεργαστή απαιτεί τον ίδιο χρόνο. Έτσι, το συνολικό κόστος επικοινωνίας κάθε επανάληψης είναι $\Theta(\log p)$. Ο παράλληλος χρόνος εκτέλεσης αυτής της υλοποίησης δίνεται από

$$T_p = \overbrace{\Theta\left(\frac{n^2}{p}\right)}^{\text{υπολογισμός}} + \overbrace{\Theta(n \log p)}^{\text{επικοινωνία}}$$

Δεδομένου ότι ο ακολουθιακός χρόνος εκτέλεσης είναι $W = \Theta(n^2)$, η επιτάχυνση και η αποδοτικότητα είναι οι εξής:

$$S = \frac{\Theta(n^2)}{\Theta\left(\frac{n^2}{p}\right) + \Theta(n \log p)}$$

$$E = \frac{1}{1 + \Theta\left(\frac{p \log p}{n}\right)} \quad (1)$$

Από την Εξίσωση (1) βλέπουμε ότι για μια βέλτιστη ως προς το κόστος παράλληλη υλοποίηση, χρειαζόμαστε χρόνο $\frac{p \log p}{n} = O(1)$. Έτσι, η υλοποίηση σε υπερκύβο του αλγορίθμου του Prim μπορεί να χρησιμοποιήσει μόνο $p = O(n / \log n)$ επεξεργαστές. Επιπλέον, από την Εξίσωση (1), η συνάρτηση ισο-αποδοτικότητας λόγω επικοινωνίας είναι $\Theta(p^2 \log^2 p)$. Δεδομένου ότι το n πρέπει να αυξάνεται τουλάχιστον τόσο γρήγορα όσο το p σε αυτή τη υλοποίηση, η συνάρτηση ισο-αποδοτικότητας λόγω ταυτόχρονης εκτέλεσης είναι $\Theta(p^2)$. Έτσι, η συνολική ισοαποδοτικότητα αυτής της υλοποίησης είναι $\Theta(p^2 \log^2 p)$.

Πλέγμα

Για ένα πλέγμα επεξεργαστών p , η εύρεση ενός ολικού ελάχιστου και η εκτέλεση μιας μετάδοσης ενός προς όλους απαιτούν χρόνο $\Theta(\sqrt{p})$. Επομένως, ο παράλληλος χρόνος εκτέλεσης του αλγορίθμου Prim είναι

$$T_p = \overbrace{\Theta\left(\frac{n^2}{p}\right)}^{\text{υπολογισμός}} + \overbrace{\Theta(n\sqrt{p})}^{\text{επικοινωνία}} \quad (2)$$

Δεδομένου ότι ο ακολουθιακός χρόνος εκτέλεσης είναι $\Omega = \Theta(n^2)$, η επιτάχυνση και η αποδοτικότητα είναι οι εξής:

$$S = \frac{\Theta(n^2)}{\Theta\left(\frac{n^2}{p}\right) + \Theta(n\sqrt{p})}$$

$$E = \frac{1}{1 + \Theta\left(\frac{p^{1.5}}{n}\right)} \quad (3)$$

Στη Εξίσωση (3) βλέπουμε ότι για έναν αλγόριθμο βέλτιστου κόστους, $p^{1.5}/n = O(1)$. Επομένως, μόνο $O(n^{0.66})$ επεξεργαστές μπορούν να χρησιμοποιηθούν αποτελεσματικά. Επίσης από την Εξίσωση (3), η συνάρτηση ισοαποδοτικότητας λόγω επικοινωνίας είναι $\Theta(p^3)$. Αυτή είναι και η συνολική συνάρτηση ισοαποδοτικότητας.

3 Συντομότερες Διαδρομές από Μία Πηγή: Ο αλγόριθμος του Dijkstra

Για ένα γράφημα με βάρη $G = (V, E, w)$, το πρόβλημα των συντομότερων διαδρομών από μία πηγή είναι η εύρεση της συντομότερης διαδρομής από μια κορυφή $v \in V$ προς όλες τις άλλες κορυφές του V . Μια συντομότερη διαδρομή από u προς v είναι μια διαδρομή ελάχιστου βάρους. Ανάλογα με την εφαρμογή, τα βάρη των ακμών μπορεί να αντιπροσωπεύουν χρόνο, κόστος, ποινή, απώλεια ή οποιαδήποτε άλλη ποσότητα που αθροίζεται σωρευτικά κατά μήκος μιας διαδρομής και πρέπει να ελαχιστοποιηθεί. Στην επόμενη ενότητα παρουσιάζουμε τον αλγόριθμο του Dijkstra, ο οποίος λύνει το πρόβλημα των συντομότερων διαδρομών από μία πηγή τόσο σε κατευθυνόμενα όσο και σε μη κατευθυνόμενα γραφήματα με μη αρνητικά βάρη.

Ο αλγόριθμος του Dijkstra, που βρίσκει τις συντομότερες διαδρομές από μία κορυφή s , είναι παρόμοιος με τον αλγόριθμο του Prim για το ελάχιστο δέντρο επικάλυψης. Όπως και ο αλγόριθμος του Prim, βρίσκει σταδιακά τις συντομότερες διαδρομές από το s προς τις υπόλοιπες κορυφές του G . Είναι επίσης άπληστος. Δηλαδή, σε κάθε βήμα επιλέγει μια κορυφή που φαίνεται ότι βρίσκεται πλησιέστερα προς την πηγή. Ο Αλγόριθμος 2 δείχνει τον αλγόριθμο του Dijkstra. Συγκρίνοντας αυτόν τον αλγόριθμο με τον αλγόριθμο του Prim για το ελάχιστο δέντρο επικάλυψης, παρατηρούμε ότι οι δύο είναι σχεδόν ταυτόσημοι. Η βασική διαφορά είναι ότι, για κάθε κορυφή $u \in (V - V_T)$, ο αλγόριθμος του Dijkstra αποθηκεύει $l[u]$, το ελάχιστο κόστος για να φτάσει στην κορυφή u από την κορυφή s χρησιμοποιώντας κορυφές στο V_T . Ο αλγόριθμος του Prim αποθηκεύει $d[u]$, το κόστος της ακμής ελάχιστου κόστους που συνδέει μια κορυφή στο V_T με την u . Ο χρόνος εκτέλεσης του αλγορίθμου του Dijkstra είναι $\Theta(n^2)$.

```

1: procedure DIJKSTRA_SINGLE_SOURCE_SP( $V, E, w, s$ )
2:    $V_T \leftarrow \{s\}$ 
3:    $l[s] \leftarrow 0$ 
4:   for all  $v \in (V - V_T)$  do
5:     if edge  $(s, v)$  exists then
6:        $l[v] \leftarrow w(s, v)$ 
7:     else
8:        $l[v] \leftarrow \infty$ 
9:     end if
10:  end for
11:  while  $V_T \neq V$  do
12:    find vertex  $u$  such that  $l[u] = \min\{l[v] \mid v \in (V - V_T)\}$ 
13:     $V_T \leftarrow V_T \cup \{u\}$ 
14:    for all  $v \in (V - V_T)$  do
15:       $l[v] \leftarrow \min\{l[v], l[u] + w(u, v)\}$ 
16:    end for
17:  end while
18: end procedure

```

Αλγόριθμος 2: Ο σειριακός αλγόριθμος του Dijkstra για τις συντομότερες διαδρομές από μία πηγή.

3.1 Παράλληλη Υλοποίηση

Η παράλληλη υλοποίηση του αλγορίθμου του Dijkstra για τις συντομότερες διαδρομές από μία πηγή είναι πολύ παρόμοια με την παράλληλη υλοποίηση του αλγορίθμου του Prim για τα ελάχιστα δέντρα επικάλυψης (Ενότητα 2). Ο πίνακας γειτνίασης με βάρη διαιρείται χρησιμοποιώντας την *block-striped* απεικόνιση. Κάθε ένας από τους p επεξεργαστές ανατίθεται σε n/p διαδοχικές στήλες του πίνακα γειτνίασης με βάρη και υπολογίζει n/p τιμές του πίνακα l . Κατά τη διάρκεια κάθε επανάληψης, όλοι οι επεξεργαστές εκτελούν υπολογισμούς και επικοινωνία ανάλογη με εκείνη που εκτελείται στην παράλληλη υλοποίηση του αλγορίθμου του Prim. Ο Πίνακας 1 παρουσιάζει τις μετρικές απόδοσης για τους αλγορίθμους του Prim και του Dijkstra σε συστήματα συνδεδεμένα ως δακτύλιος, πλέγμα και υπερκύβος. Σε όλες τις περιπτώσεις, ο παράλληλος χρόνος εκτέλεσης για μια βέλτιστη υλοποίηση ως προς το κόστος είναι επίσης ο ελάχιστος παράλληλος χρόνος εκτέλεσης που μπορεί να επιτευχθεί σε κάθε αρχιτεκτονική σε ασυμπτωτικούς όρους.

Πίνακας 1: Επιδόσεις των αλγορίθμων Prim και Dijkstra σε διαφορετικές αρχιτεκτονικές.

Αρχιτεκτονική	Μέγιστος αριθμός επεξεργαστών για $E = \Theta(1)$	Αντίστοιχος παράλληλος χρόνος	Συνάρτηση ισοαποδοτικότητας
Δακτύλιος	$\Theta(\sqrt{n})$	$\Theta(n^{1.5})$	$\Theta(p^4)$
Πλέγμα	$\Theta(n^{0.66})$	$\Theta(n^{1.33})$	$\Theta(p^3)$
Υπερκύβος	$\Theta\left(\frac{n}{\log n}\right)$	$\Theta(n \log n)$	$\Theta(p^2 \log^2 p)$

3.1.1 API/Σύστημα μεταβίβασης μηνυμάτων

Το API μεταβίβασης μηνυμάτων πρέπει να είναι διαθέσιμο σε όλα τα συστήματα στα οποία υλοποιείται και θα πρέπει να είναι όσο το δυνατόν πιο απλό και ευθύ, προτιμητέα να υποστηρίζει συλλογικές λειτουργίες. Το MPI καλύπτει αυτές τις απαιτήσεις, επομένως δεν απαιτείται σοβαρή εξέταση εναλλακτικών κατά την επιλογή του. Η υλοποίηση του MPI είναι δωρεάν, εύκολα διαθέσιμη, και παρέχει συνδέσεις (bindings) για τη γλώσσα C που είναι ευρέως γνωστή.

3.1.2 API/Σύστημα κοινής διεύθυνσης

Το επιλεγμένο API κοινής διεύθυνσης πρέπει να επιτρέπει μεγάλο έλεγχο πάνω στις εργασίες που εκτελούνται ταυτόχρονα. Πρέπει επίσης να παρέχεται δωρεάν. Το OpenMP παρέχει ένα επίπεδο πάνω από τις εγγενείς νηματικές (thread) λειτουργίες για τη διευκόλυνση διαφόρων εργασιών σχετικών με νήματα. Χρησιμοποιώντας τις οδηγίες που παρέχει το OpenMP, ο προγραμματιστής δεν χρειάζεται να εκτελέσει την εργασία αρχικοποίησης του αντικειμένου ιδιοτήτων, να ορίσει παραμέτρους για το νήμα και να διαιρέσει τον χώρο επανάληψης. Αυτή η δυνατότητα είναι ιδιαίτερα χρήσιμη όταν το υποκείμενο πρόβλημα έχει στατικό ή τακτικό διάγραμμα εργασιών. Στο πλαίσιο διαφόρων εφαρμογών, έχει αποδειχθεί ότι το κόστος που σχετίζεται με την αυτόματη δημιουργία κώδικα νημάτων από αυτές τις οδηγίες είναι ελάχιστο.

3.1.3 Γλώσσα

Επιλέγεται τη γλώσσα C λόγω της διαθεσιμότητας μιας σχετικά σταθερής υλοποίησης του MPI για μεταβίβαση μηνυμάτων και της βιβλιοθήκης για OpenMP.

3.1.4 Περιβάλλον συστήματος

Η σειριακή υλοποίηση του αλγορίθμου του Dijkstra, η υλοποίηση του αλγορίθμου του Dijkstra με MPI και η υλοποίηση του αλγορίθμου του Dijkstra με OpenMP, και οι τρεις υλοποιήσεις εκτελούνται στο σύστημα του Minnesota Supercomputing Institute (MSI).

3.2 Δεδομένα δοκιμών

Αφού ορίστηκαν οι τεχνολογίες που θα χρησιμοποιηθούν, χρειάζεται ένα πρόγραμμα σε JAVA για τη δημιουργία δοκιμαστικών δεδομένων. Το πρόγραμμα αυτό παράγει έναν δισδιάστατο πίνακα με τυχαίους αριθμούς στο διάστημα 1 έως 15, που αντιπροσωπεύει το γράφημα εισόδου για τον αλγόριθμο του Dijkstra. Τα βάρη δημιουργούνται με τη συνάρτηση Random της JAVA. Έστω G ένας δισδιάστατος πίνακας, όπου $G[i][j]$ αντιπροσωπεύει το βάρος από την κορυφή i προς την κορυφή j . Εάν δεν υπάρχει άμεση σύνδεση, το βάρος ορίζεται σε 9999999, αλλιώς είναι ένας τυχαίος αριθμός μεταξύ 1 και 15. Όταν $i = j$, πρόκειται για την ίδια κορυφή, οπότε ορίζουμε $G[i][j] = 0$ σε αυτή την περίπτωση. Τα βάρη παράγονται τυχαία, και η υπολογιζόμενη απόσταση δεν θα είναι πολύ μεγάλη. Αυτό δεν επηρεάζει τους πειραματικούς μας στόχους, εφόσον χρειαζόμαστε μόνο τα αποτελέσματα από διαφορετικά προγράμματα που χρησιμοποιούν τα ίδια δεδομένα. Τρέχουμε το ίδιο σύνολο δεδομένων στη σειριακή υλοποίηση του αλγορίθμου του Dijkstra, στην υλοποίηση του αλγορίθμου του

Dijkstra με MPI και στην υλοποίηση του αλγορίθμου του Dijkstra με OpenMP για να συγκρίνουμε τους χρόνους εκτέλεσης. Συνολικά χρησιμοποιούνται έξι σύνολα δεδομένων ως πίνακες γειτνίασης. Αυτό σημαίνει για γραφήματα με 8, 64, 256, 512, 1024 και 2048 κορυφές. Ακολουθεί ένα παράδειγμα για τη μήτρα των 8 κορυφών.

0	2	9999999	3	4	3	9999999	3
2	0	8	8	9	9999999	7	7
9999999	8	0	6	7	9999999	9999999	2
3	8	6	0	7	3	9	7
4	9	7	7	0	9999999	9999999	4
3	9999999	9999999	3	9999999	0	8	3
9999999	7	9999999	9	9999999	8	0	9999999
3	7	2	7	4	3	9999999	0

3.3 Αλγόριθμος

Στην παρακάτω ενότητα δίνονται λεπτομέρειες σχετικά με το πώς υλοποιήθηκε ο αλγόριθμος. Εδώ παρέχεται ψευδοκώδικας που περιγράφει τις υλοποιήσεις σε απλοποιημένη μορφή.

3.3.1 Υλοποίηση του αλγορίθμου του Dijkstra με MPI

Η υλοποίηση του αλγορίθμου, σε απλοποιημένη μορφή σύμφωνα με τον ψευδοκώδικα, παρουσιάζεται παρακάτω.

```

1: procedure DIJKSTRA_SINGLE_SOURCE_SP_MPI( $V, E, wgt, lengths, s$ )
2:   initialize local and global data structures
3:   for all  $v \in V$  do
4:     set  $lminpair[0]$ : local min distance
5:     set  $lminpair[1]$ : corresponding vertex
6:   end for
7:   while not all vertices processed do
8:     for vertices in each processor find a vertex  $u$  with the smallest distance from
       the source  $s$ 
9:     perform MPI_Allreduce to obtain the global minimum across processors
10:    get the global minimum vertex  $u$  and mark it as processed
11:    for all  $v \in nlocals$  do ▷  $nlocals$  = number of vertices stored locally
12:       $lengths[v] \leftarrow \min\{lengths[v], udist + wgt[u * nlocal + v]\}$ 
13:    end for
14:  end while
15: end procedure

```

Αλγόριθμος 3: Ο αλγόριθμος του Dijkstra με MPI για τις συντομότερες διαδρομές από μία πηγή.

Ο βασικός υπολογιστικός βρόχος του παράλληλου αλγορίθμου του Dijkstra για τις συντομότερες διαδρομές από μία πηγή εκτελεί τρία βήματα. Πρώτον, κάθε διεργασία εντοπίζει την τοπικά αποθηκευμένη κορυφή στο V_o με τη μικρότερη απόσταση από την πηγή. Έπειτα, οι διεργασίες καθορίζουν την κορυφή με τη μικρότερη απόσταση και την προσθέτουν στο V_c . Τρίτον, κάθε διεργασία ενημερώνει τον πίνακα αποστάσεων ώστε να ληφθεί υπόψη ότι στο V_c έχουν προστεθεί νέες κορυφές.

Το πρώτο βήμα είναι η σάρωση των τοπικών κορυφών του V_o για να βρεθεί η κορυφή v με τη μικρότερη τοπική απόσταση $[v]$. Το αποτέλεσμα του υπολογισμού αποθηκεύεται στον πίνακα $lminpair$. Συγκεκριμένα, ο $lminpair[0]$ αποθηκεύει την απόσταση μεταξύ των κορυφών και ο $lminpair[1]$ αποθηκεύει τις ίδιες τις κορυφές. Ακολουθώντας τα παρακάτω βήματα θα γίνει κατανοητό γιατί θα πρέπει να χρησιμοποιηθεί αυτή η λύση αποθήκευσης. Το επόμενο βήμα είναι ο προσδιορισμός της κορυφής με τη μικρότερη συνολική απόσταση προς την πηγή, το οποίο γίνεται ελαχιστοποιώντας την τιμή που περιέχεται στο $lminpair[0]$.

Ωστόσο, πέρα από την ίδια την ελάχιστη απόσταση, χρειάζεται και ο δείκτης (η ταυτότητα) της κορυφής που επιτυγχάνει αυτή την απόσταση. Γι' αυτό η κατάλληλη λειτουργία μείωσης είναι η MPI_MINLOC , η οποία επιστρέφει την ελάχιστη τιμή και τον δείκτη που αντιστοιχεί στην ελάχιστη αυτή τιμή. Εξαιτίας της MPI_MINLOC χρησιμοποιούμε πίνακα δύο στοιχείων $lminpair$ για να αποθηκεύσουμε την απόσταση και την κορυφή που την επιτυγχάνει. Επιπλέον, όλες οι διεργασίες χρειάζονται το αποτέλεσμα της διαδικασίας ανάκτησης δεδομένων για να εκτελέσουν το τρίτο βήμα, γι' αυτό χρησιμοποιούμε την $MPI_Allreduce$ για να πραγματοποιήσουμε τη μείωση. Το αποτέλεσμα της μείωσης επιστρέφεται στον πίνακα $gminpair$. Το τρίτο και τελευταίο βήμα κάθε επανάληψης εκτελείται σαρώνοντας τις τοπικές κορυφές που ανήκουν στο V_o και ενημερώνοντας, για κάθε τοπική κορυφή v , την ελάχιστη απόσταση μεταξύ αυτής και της κορυφής-πηγής.

Στο πρόγραμμα MPI, γίνεται ανάθεση σε κάθε επεξεργαστή n/p διαδοχικές στήλες του πίνακα W και χρησιμοποιείται η μείωση MPI_MINLOC για να επιλεγεί η κορυφή v που θα συμπεριληφθεί στο V_c σε κάθε επανάληψη. Αξίζει να σημειωθεί ότι ο δείκτης που επιστρέφει η MPI_MINLOC σε συγκρίσεις όπως (a, i) και (a, j) είναι ο μικρότερος δείκτης όταν οι

τιμές είναι ίδιες. Επομένως, μεταξύ των κορυφών που έχουν την ίδια ελάχιστη απόσταση προς την πηγή, προτιμώνται οι κορυφές με τον μικρότερο δείκτη. Αυτό μπορεί να οδηγήσει σε ανισορροπία φόρτου, αφού οι κορυφές που αποθηκεύονται σε διεργασίες με μικρότερους δείκτες τείνουν να περιλαμβάνονται πρώτες στο V_c σε σχέση με αυτές που βρίσκονται σε διεργασίες με μεγαλύτερους δείκτες (ειδικά όταν πολλές κορυφές στο V_o έχουν την ίδια μικρή απόσταση προς την πηγή). Συνεπώς, στις διεργασίες με μεγαλύτερους δείκτες το διαμορφωμένο μέγεθος του V_o θα είναι μεγαλύτερο και ο συνολικός χρόνος εκτέλεσης μπορεί να κυριαρχείται από αυτές.

Μια λύση σε αυτό το πρόβλημα είναι η κυκλική κατανομή των στηλών του W . Αυτή η διαδικασία κατανομής συγκεντρώνει όλες τις p κορυφές, ξεκινώντας από την κορυφή i . Σε αυτό το πλαίσιο, κάθε διεργασία εξακολουθεί να αναθέτει n/p κορυφές αλλά οι δείκτες τους καλύπτουν σχεδόν ολόκληρο το γράφημα. Έτσι, η *MPI_MINLOC* θα ευνοεί συστηματικά κορυφές με μικρότερους δείκτες και δεν θα προκαλείται ανισορροπία φόρτου [2].

3.3.2 Υλοποίηση του αλγορίθμου του Dijkstra με OpenMP

Οι περισσότερες δομές του OpenMP εφαρμόζονται σε ένα δομημένο μπλοκ, δηλαδή σε ένα μπλοκ μιας ή περισσότερων εντολών με ένα σημείο εισόδου στην κορυφή και ένα σημείο εξόδου στο κάτω μέρος. Καθίσταται εφικτός ο εντοπισμός των υπολογιστικά εντατικών βρόχων στον σειριακό αλγόριθμο του Dijkstra και οι επαναλήψεις του βρόχου γίνονται ανεξάρτητες, ενώ στη συνέχεια μπορεί να γίνει η τοποθέτηση των κατάλληλων οδηγιών του OpenMP και να γίνουν δοκιμές.

```

1: procedure DIJKSTRA_SP_OMP( $V, E, w, distances, s$ )
2:   begin
3:     while not all vertices processed do
4:       #pragma omp parallel private
5:       shared()
6:       omp_get_thread_num()
7:       omp_get_num_threads()
8:       Each thread finds the min distance  $u$  unconnected vertex inner
9:       #pragma omp critical // update overall min
10:      #pragma omp barrier
11:      #pragma omp single // mark new vertex as done
12:      for all  $v$  in each thread do
13:         $distances[v] \leftarrow \min\{distances[v], distances[u] + w[u][v]\}$ 
14:      end for
15:    end while
16: end procedure

```

Αλγόριθμος 4: Ο αλγόριθμος του Dijkstra με OpenMP για τις συντομότερες διαδρομές από μία πηγή.

Όπως δείχνει ο ψευδοκώδικας, η υλοποίηση του αλγορίθμου Dijkstra με OpenMP είναι πολύ παρόμοια με τη σειριακή. Σε σύγκριση με την υλοποίηση με MPI, η υλοποίηση με OpenMP απαιτεί λιγότερες γραμμές κώδικα. Η συνάρτηση *omp_set_num_threads* ορίζει τον προεπιλεγμένο αριθμό νημάτων που θα δημιουργηθούν όταν συναντηθεί η επόμενη οδηγία *parallel*. Χρησιμοποιούμε αυτή τη συνάρτηση στη συνάρτηση *main*. Η

omp_get_num_threads επιστρέφει τον αριθμό των νημάτων που συμμετέχουν σε μια ομάδα. Η *omp_get_thread_num* επιστρέφει ένα μοναδικό αναγνωριστικό για κάθε νήμα στην ομάδα. Αυτός ο ακεραίος παίρνει τιμές από 0 έως *omp_get_num_threads()* - 1. Η οδηγία *critical* διασφαλίζει ότι, σε κάθε χρονική στιγμή κατά την εκτέλεση, μόνο ένα νήμα βρίσκεται μέσα σε μια κρίσιμη ενότητα προσδιορισμένη με ένα συγκεκριμένο όνομα. Η OpenMP παρέχει την οδηγία *critical* για την υλοποίηση κρίσιμων περιοχών. Μια κρίσιμη περιοχή επιτρέπει σε διαφορετικά νήματα να εκτελούν διαφορετικό κώδικα ενώ είναι προστατευμένα μεταξύ τους. Το *barrier* είναι μία από τις πιο συχνά χρησιμοποιούμενες πρωτόγονες λειτουργίες συγχρονισμού. Η OpenMP παρέχει την οδηγία *barrier*. Όταν η εκτέλεση συναντήσει αυτή την οδηγία, όλα τα νήματα της ομάδας περιμένουν μέχρι να φτάσουν τα υπόλοιπα και στη συνέχεια συνεχίζουν. Η οδηγία *single* προσδιορίζει ένα δομημένο μπλοκ που εκτελείται από ένα μόνο νήμα. Κατά την είσοδο στο μπλοκ *single*, το πρώτο νήμα εισέρχεται και όλα τα υπόλοιπα νήματα προχωρούν μέχρι το τέλος του μπλοκ [2].

3.4 Αποτελέσματα Υλοποίησης και Ανάλυση

Ο Πίνακας 2 περιέχει όλα τα βασικά αποτελέσματα από την σειριακή εκτέλεση του αλγορίθμου του Dijkstra, τα MPI και OpenMP προγράμματα. Ο κώδικας εκτελέστηκε στο ίδιο περιβάλλον συστήματος και τα δεδομένα εισόδου δημιουργήθηκαν με τη συνάρτηση *Random*. Στην παρούσα εργασία χρησιμοποιούνται συνολικά έξι σύνολα δεδομένων για την είσοδο του πίνακα γειτνίασης. Συγκεκριμένα, για το γράφημα χρησιμοποιούνται 8 κορυφές, 64 κορυφές, 256 κορυφές, 512 κορυφές, 1024 κορυφές και 2048 κορυφές. Μετά την εκτέλεση του κώδικα, λαμβάνονται τα αποτελέσματα ως διάρκεια σε δευτερόλεπτα. Για τους παράλληλους υπολογισμούς με OpenMP και MPI, χρησιμοποιήθηκαν 2, 4, 8, 16, 32, 64, 128, 256 και 512 επεξεργαστές (ο αριθμός των κορυφών πρέπει να είναι μεγαλύτερος από τον αριθμό των επεξεργαστών) για την εκτέλεση του κώδικα.

Πίνακας 2: Χρόνος εκτέλεσης σε δευτερόλεπτα και των τριών υλοποιήσεων

	8 κορυφές	64 κορυφές	256 κορυφές	512 κορυφές	1024 κορυφές	2048 κορυφές
<i>Seq</i>	0.0035	0.0029	0.0099	0.1162	0.3568	0.9965
<i>OpenMP2</i>	0.0003	0.0015	0.0093	0.0651	0.2493	0.8583
<i>OpenMP4</i>	0.0003	0.0016	0.0109	0.0635	0.2488	0.7933
<i>OpenMP8</i>	0.0007	0.8831	0.0140	0.0854	0.2807	0.9432
<i>OpenMP16</i>		0.0574	0.0233	0.2736	0.6794	1.7490
<i>OpenMP32</i>		0.0640	0.1186	0.5117	1.3014	2.8708
<i>OpenMP64</i>		0.0864	0.2030	0.7614	1.7558	3.5983
<i>OpenMP128</i>			0.3302	1.3960	2.8929	5.9080
<i>OpenMP256</i>			0.9902	2.7855	5.3655	10.5335
<i>OpenMP512</i>				6.1405	10.6755	20.9920
<i>MPI2</i>	0.0246	0.0178	0.0287	0.1137	0.1316	0.5040
<i>MPI4</i>	0.0353	0.0210	0.0218	0.0413	0.1281	0.4723
<i>MPI8</i>	0.0144	0.0164	0.0264	0.0440	0.1324	1.3663
<i>MPI16</i>		0.0264	0.0453	0.1010	0.2055	3.3525
<i>MPI32</i>		10.8733	1.2415	56.9774	3.9622	21.4067
<i>MPI64</i>		27.2983	3.1004	67.9168	7.1446	41.7286
<i>MPI128</i>			7.3566	19.8329	17.9661	119.9792
<i>MPI256</i>			18.8646	32.8870	44.8353	343.0892
<i>MPI512</i>				488.7678	1733.9085	2399.9986

Πίνακας 3: Ο καλύτερος χρόνος εκτέλεσης σε δευτερόλεπτα και των τριών υλοποιήσεων (οι αριθμοί στις παρενθέσεις δείχνουν το πλήθος των νημάτων/επεξεργαστών

	8 κορυφές	64 κορυφές	256 κορυφές	512 κορυφές	1024 κορυφές	2048 κορυφές
<i>Seq</i>	0.0035	0.0029	0.0099	0.1162	0.3568	0.9965
<i>OpenMP</i>	0.0003(2)	0.0015(2)	0.0093(2)	0.0635(4)	0.2488(4)	0.7933(4)
<i>MPI</i>	0.0144(8)	0.0164(8)	0.0218(4)	0.0413(4)	0.1281(4)	0.4723(4)



Σχήμα 7: Ο καλύτερος χρόνος εκτέλεσης για κάθε υλοποίηση σε διαφορετικά σύνολα δεδομένων

Το Σχήμα 7 δείχνει τον καλύτερο χρόνο εκτέλεσης για τις υλοποιήσεις σειριακά, με OpenMP και με MPI για διαφορετικό αριθμό κορυφών. Γίνεται αντιληπτό ότι η απόδοση είναι καλύτερη όταν η εκτέλεση του αλγορίθμου γίνεται με OpenMP ή MPI. Για μικρό αριθμό κορυφών, μπορεί να δαπανάται περισσότερος χρόνος σε παραλληλοποίηση και συγχρονισμό απ' ό,τι στην εκτέλεση του σειριακού κώδικα. Επομένως, όταν ο αριθμός κορυφών είναι μικρότερος των 512, δεν είναι προφανές ότι η παραλληλοποίηση υπερέχει της σειριακής ε-

κτέλεσης. Προβλέπεται ότι ο χρόνος εκτέλεσης με MPI και OpenMP θα είναι σημαντικά καλύτερος σε σχέση με τον σειριακό για μεγαλύτερο αριθμό κορυφών.

Ένα ακόμη αποτέλεσμα είναι ότι ο καλύτερος χρόνος εκτέλεσης για το MPI είναι ελαφρώς καλύτερος από τον OpenMP καθώς αυξάνεται ο αριθμός των κορυφών. Σε ένα σύστημα με κοινό χώρο διευθύνσεων, κάθε φορά που ένας επεξεργαστής χρειάζεται να διαβάσει δεδομένα που έχει γράψει άλλος επεξεργαστής, η προσωρινή μνήμη του πρέπει να ενημερωθεί. Όταν πολλοί επεξεργαστές διαβάζουν και γράφουν δεδομένα στην ίδια γραμμή της προσωρινής μνήμης, η προσωρινή μνήμη πρέπει να ενημερώνεται συνεχώς. Αυτό σημαίνει ότι η προσωρινή μνήμη δεν είναι αποτελεσματική καθώς πρέπει να ενημερώνεται διαρκώς. Αυτό μπορεί να έχει μεγάλο αντίκτυπο στην απόδοση αλγορίθμων σε συστήματα με κοινό χώρο διευθύνσεων. Αντίθετα, στα κατανεμημένα συστήματα που χρησιμοποιούν ανταλλαγή μηνυμάτων, κάθε επεξεργαστής διαθέτει ξεχωριστή προσωρινή μνήμη η οποία δεν αδρανοποιείται ή ενημερώνεται άμεσα από άλλους επεξεργαστές. Επομένως, η συνοχή της προσωρινής μνήμης δεν αποτελεί τόσο μεγάλο πρόβλημα στα συστήματα ανταλλαγής μηνυμάτων.

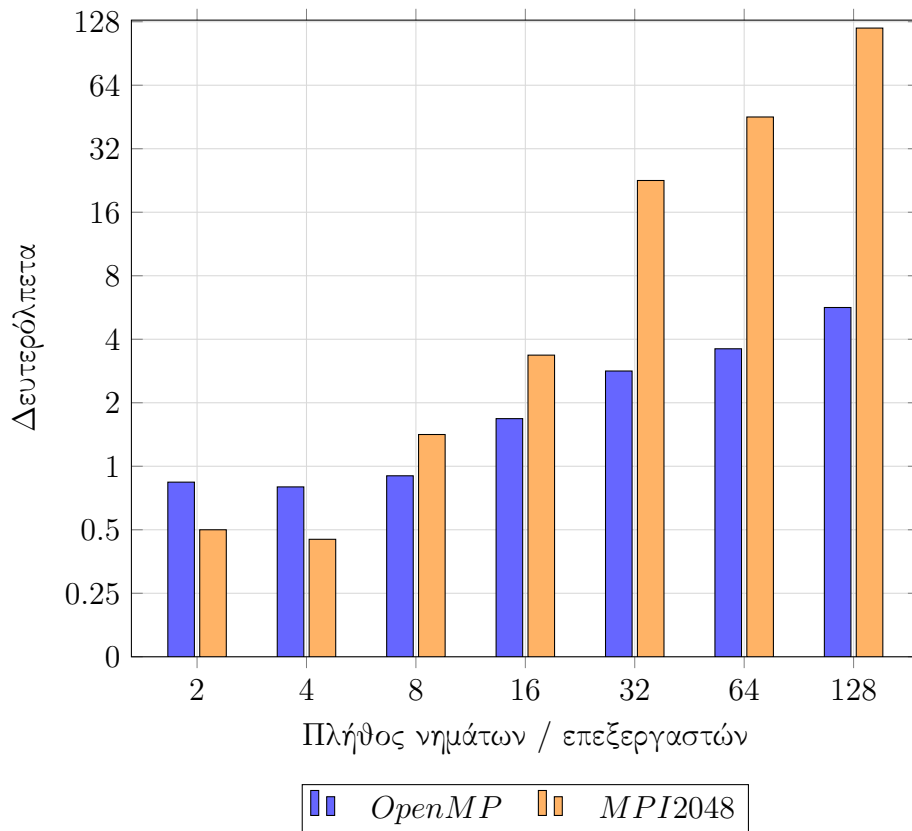
Ο παρακάτω πίνακας δείχνει τον χρόνο υπολογισμού για OpenMP και MPI όταν ο αριθμός κορυφών είναι 1024 και 2048.

Πίνακας 4: Τα αποτελέσματα για το OpenMP και το MPI για διαφορετικά νήματα/επεξεργαστές

πλήθος νημάτων / επεξεργασ- τών	<i>OpenMP</i> 1024 κορυφές	<i>MPI</i> 1024 κορυφές	<i>OpenMP</i> 2048 κορυφές	<i>MPI</i> 2048 κορυφές
2	0.2493	0.1316	0.8583	0.5040
4	0.2488	0.1281	0.7933	0.4723
8	0.2807	0.1324	0.9432	1.3663
16	0.6794	0.2055	1.7490	3.3525
32	1.3014	3.9622	2.8708	21.4067
64	1.7558	7.1446	3.5983	41.7286
128	2.8929	17.9661	5.9080	119.9792



Σχήμα 8: Διαφορετικά νήματα/επεξεργαστές για την υλοποίηση *OpenMP* και *MPI* σε γράφημα με 1024 κορυφές



Σχήμα 9: Διαφορετικά νήματα/επεξεργαστές για την υλοποίηση *OpenMP* και *MPI* σε γράφημα με 2048 κορυφές

Το Σχήμα 8 και το Σχήμα 9 παρουσιάζουν την απόδοση του αλγορίθμου του Dijkstra σε διαφορετικούς σχηματισμούς επεξεργαστών/νημάτων. Σύμφωνα με τον Πίνακα 2, είναι γνωστό ότι οι υλοποιήσεις με *OpenMP* και *MPI* αποδίδουν καλύτερα από τις σειριακές υλοποιήσεις για τον αλγόριθμο του Dijkstra. Ωστόσο, από τα Σχήματα 8 και 9 γίνεται αντιληπτό ότι ο μικρότερος αριθμός επεξεργαστών/νημάτων δίνει το ταχύτερο αποτέλεσμα. Για παράδειγμα, αν χρησιμοποιηθούν 1024 κορυφές, ο βέλτιστος αριθμός επεξεργαστών για την υλοποίηση *MPI* είναι 2, 4, 8. Για την *OpenMP* ο αριθμός είναι ο ίδιος. Όταν ο αριθμός των κορυφών είναι 2048, καταλήγουμε σε παρόμοιο συμπέρασμα. Αυτό πιθανώς οφείλεται στο ότι κάθε πρόσθετη διεργασία/νήμα στον κώδικα προκαλεί επιπλέον κόστη επικοινωνίας κατά την ενημέρωσή τους. Καθώς ο αριθμός των επεξεργαστών/νημάτων αυξάνεται εμφανώς, αυτά τα κόστη επικοινωνίας αποκτούν σημαντική επίπτωση. Ειδικότερα για την *MPI* υλοποίηση του αλγορίθμου του Dijkstra, η απόδοση είναι πολύ κακή σε σύγκριση με την αντίστοιχη του *OpenMP*, και η αύξηση του αριθμού των επεξεργαστών εντείνει την επιβράδυνση. Η παράλληλη απόδοση πιθανόν είναι πολύ κακή, διότι κυριαρχείται από τον χρόνο επικοινωνίας, δηλαδή τον χρόνο που απαιτείται για την εκτέλεση του *MPI_Allreduce* σε κάθε επανάληψη.

Σύμφωνα με την παράλληλη υλοποίηση του αλγορίθμου του Dijkstra, είναι δυνατόν να υπολογιστεί η επιτάχυνση σε κάθε συνθήκη. Ο ακόλουθος πίνακας συγκρίνει την θεωρητική επιτάχυνση με την πειραματική επιτάχυνση στην *MPI* υλοποίηση για είσοδο σε γράφημα με 1024 κορυφές.

Πίνακας 5: Η σύγκριση της θεωρητικής επιτάχυνσης με την πειραματική επιτάχυνση για την υλοποίηση MPI για γράφημα 1024 κορυφών και διαφορετικό αριθμό επεξεργαστών

Πλήθος επεξεργαστών	Θεωρητική επιτάχυνση για 1024 κορυφές	Πειραματική επιτάχυνση για 1024 κορυφές
2	1.9961	2.7112
4	3.9690	2.7853
8	7.8168	2.6947
16	15.0588	1.7363
32	27.6757	0.0901
64	46.5455	0.0499
128	68.2667	0.0199



Σχήμα 10: Η σύγκριση της θεωρητικής επιτάχυνσης με την πειραματική επιτάχυνση για την υλοποίηση MPI για γράφημα 1024 κορυφών σε διαφορετικούς επεξεργαστές

Η επιτάχυνση είναι μέτρο που αποτυπώνει το σχετικό όφελος της επίλυσης ενός προβλήματος σε παραλληλία. Ορίζεται ως ο λόγος του χρόνου που απαιτείται για την επίλυση ενός προβλήματος με τον καλύτερο σειριακό αλγόριθμο προς τον χρόνο που απαιτείται για την επίλυση του ίδιου προβλήματος σε έναν παράλληλο υπολογιστή με p όμοια στοιχεία επεξεργασίας. Αν η επιτάχυνση μπορεί να διατηρήσει γραμμική αύξηση με τους επεξεργαστές, πολλαπλές μηχανές μπορούν να μειώσουν σημαντικά τον απαιτούμενο χρόνο. Ωστόσο, αυτή η επιτάχυνση είναι πολύ δύσκολο να επιτευχθεί, γιατί με την αύξηση του πλήθους των μηχανών προκύπτουν προβλήματα απωλειών επικοινωνίας, καθώς και προβλήματα από τους

ιδίους τους κόμβους (Διαφορά ρυθμού χρονισμού - the skew of the slaves). Για παράδειγμα, ο συνολικός χρόνος εκτέλεσης του αλγορίθμου καθορίζεται συνήθως από την πιο αργή μηχανή. Αν ο χρόνος που απαιτείται από κάθε υπολογιστή διαφέρει, τότε υπάρχει το πρόβλημα της διαφοράς ρυθμού χρονισμού.

Υπάρχουν πολλές πιθανές αιτίες για τις οποίες το υψηλό επίπεδο παραλληλίας στην υλοποίηση του Dijkstra δεν επέφερε την αναμενόμενη επιτάχυνση. Μία αιτία μπορεί να είναι ότι ο χρησιμοποιούμενος κώδικας είναι αναποτελεσματικός.

Στο πείραμα, η μείωση της επιτάχυνσης μπορεί επίσης να οφείλεται στο ότι η λανθάνουσα καθυστέρηση επικοινωνίας υπερσχύει του οφέλους από τη χρήση περισσότερων επεξεργαστών. Πρέπει να ληφθούν υπόψη όλες οι πληροφορίες που απαιτούνται για την αξιολόγηση της απόδοσης ενός παράλληλου αλγορίθμου σε μια συγκεκριμένη αρχιτεκτονική με συγκεκριμένες τεχνολογικά εξαρτώμενες σταθερές, όπως η ταχύτητα της Κεντρικής Μονάδας Επεξεργασίας (ΚΜΕ - CPU) και η ταχύτητα επικοινωνίας.

3.5 Συμπεράσματα και Περαιτέρω Έρευνα

Σε αυτή την εργασία εισήχθη, σχεδιάστηκε και υλοποιήθηκε ο παράλληλος αλγόριθμος του Dijkstra. Τα ευρήματα οδηγούν στα ακόλουθα συμπεράσματα:

- Η απόδοση του αλγορίθμου του Dijkstra είναι καλύτερη όταν χρησιμοποιούνται οι υλοποιήσεις με OpenMP και MPI σε σύγκριση με τη σειριακή υλοποίηση, ιδιαίτερα για μεγάλα σύνολα εισόδου.
- Στην παραλληλοποίηση του αλγορίθμου του Dijkstra, ο καλύτερος χρόνος εκτέλεσης για την MPI είναι ελαφρώς καλύτερος από αυτόν της OpenMP όσο αυξάνεται ο αριθμός των κορυφών.
- Και για τις δύο υλοποιήσεις OpenMP και MPI, ο μικρότερος αριθμός επεξεργαστών/νημάτων δίνει το ταχύτερο αποτέλεσμα.
- Συγκρίνοντας τη θεωρητική επιτάχυνση με την πειραματική επιτάχυνση στην υλοποίηση MPI με είσοδο 1024 κορυφών, η πειραματική επιτάχυνση δεν παρουσιάζει γραμμική αύξηση με την αύξηση των επεξεργαστών. Σε σύγκριση με τη σειριακή εκτέλεση, η παράλληλη εκτέλεση του αλγορίθμου του Dijkstra δεν εμφανίζει ικανοποιητική απόδοση όσον αφορά την επιτάχυνση.

Τα ακόλουθα θα ήταν χρήσιμα θέματα για περαιτέρω έρευνα:

- Βελτιστοποίηση των υλοποιήσεων του αλγορίθμου. Πλήρως βελτιστοποιημένες υλοποιήσεις, και ιδιαίτερα η χρήση ουράς προτεραιότητας αντί για πίνακα, αποτελούν πεδίο που επιτρέπει σημαντική περαιτέρω εργασία. Μια ουρά προτεραιότητας είναι τέτοια όπου κάθε στοιχείο έχει επιπλέον μια «προτεραιότητα». Σε μια ελάχιστη ουρά προτεραιότητας, το ελάχιστο στοιχείο έχει την υψηλότερη προτεραιότητα και εξυπηρετείται πριν από ένα στοιχείο με χαμηλότερη προτεραιότητα. Μια ελάχιστη ουρά προτεραιότητας παρέχει τρεις βασικές λειτουργίες: *add_with_priority()*, *decrease_priority()* και *extract_min()*. Έστω $|V|$ ο αριθμός κορυφών και $|E|$ ο αριθμός ακμών ενός γράφηματος. Αν ο αλγόριθμος του Dijkstra χρησιμοποιεί πίνακα για να σαρώσει άμεσα όλες τις κορυφές, το κόστος είναι $O(|V|^2)$. Για αραιά γράφημα, εάν ο αριθμός των ακμών είναι μικρότερος από τον αριθμό των κορυφών, η υλοποίηση του γραφήματος εισόδου

μπορεί να γίνει με λίστα γειτνίασης αντί για πίνακα γειτνίασης και να χρησιμοποιηθεί δυαδικός σωρός ή σωρός Fibonacci ως ουρά προτεραιότητας για βελτιστοποίηση.

Η διαδικασία είναι η εξής:

1. Προσθήκη της κορυφής πηγής στον σωρό και αναπροσαρμογή του σωρού.
2. Επιλογή του κορυφαίου στοιχείου u και διαγραφή του από τον σωρό.
3. Επεξεργασία των κορυφών που είναι γειτονικές του u : αν μια κορυφή βρίσκεται στην ουρά, ενημέρωση της απόστασης και αναπροσαρμογή της θέσης του στοιχείου στον σωρό. Αν η κορυφή δεν βρίσκεται στον σωρό, προσθήκη της στον σωρό και αναπροσαρμογή του σωρού.
4. Αν το προκύπτον u είναι το τελικό σημείο, τερματίζει ο αλγόριθμος. Αλλιώς επανάληψη των βημάτων 2 και 3.

Η πολυπλοκότητα με χρήση δυαδικού σωρού απαιτεί $O((|E| + |V|) \log |V|)$. Ο σωρός Fibonacci βελτιώνει αυτό σε $O(|E| + |V| \log |V|)$.

- Εκτέλεση πειραματικών δοκιμών σε διαφορετικά περιβάλλοντα συστήματος.

4 Συντομότερες Διαδρομές για Όλα τα Ζεύγη

Αντί να βρίσκουμε τις συντομότερες διαδρομές από μία μόνο κορυφή v προς κάθε άλλη κορυφή, μερικές φορές μας ενδιαφέρει να βρούμε τις συντομότερες διαδρομές μεταξύ όλων των ζευγών κορυφών. Συγκεκριμένα, δεδομένου ενός γραφήματος με βάρη $G = (V, E, w)$, το πρόβλημα των συντομότερων διαδρομών για όλα τα ζεύγη (**all-pairs shortest paths**) είναι να βρεθούν οι συντομότερες διαδρομές μεταξύ όλων των ζευγών κορυφών $v_i, v_j \in V$ με $i \neq j$. Για γράφημα με n κορυφές, η έξοδος ενός αλγορίθμου για όλα τα ζεύγη είναι ένας $n \times n$ πίνακας $D = (d_{i,j})$ τέτοιος ώστε $d_{i,j}$ να είναι το κόστος της συντομότερης διαδρομής από την κορυφή v_i στην κορυφή v_j .

Οι ακόλουθες ενότητες παρουσιάζουν τρεις αλγορίθμους για την επίλυση του προβλήματος των συντομότερων διαδρομών για όλα τα ζεύγη. Ο πρώτος αλγόριθμος χρησιμοποιεί τον πολλαπλασιασμό πίνακα, ο δεύτερος χρησιμοποιεί τον αλγόριθμο του Dijkstra για συντομότερες διαδρομές από μία πηγή, και ο τρίτος χρησιμοποιεί τον αλγόριθμο του Floyd. Ο αλγόριθμος του Dijkstra απαιτεί μη αρνητικά βάρη ακμών. Οι άλλοι δύο αλγόριθμοι λειτουργούν με γραφήματα που έχουν ακμές με αρνητικό βάρος εφόσον δεν περιέχουν κύκλους αρνητικού βάρους.

4.1 Αλγόριθμος βασισμένος στον Πολλαπλασιασμό Πίνακα

Έστω ένα γράφημα με βάρη $G = (V, E, w)$ που δεν περιέχει κύκλους με αρνητικό βάρος, και έστω ότι ο πίνακας γειτνίασης με βάρη είναι $A = (a_{i,j})$. Ορίζουμε το $p_{i,j}^{(k)}$ ως την ελάχιστου βάρους διαδρομή από την κορυφή v_i στην κορυφή v_j που περιέχει το πολύ k ακμές, και το $d_{i,j}^{(k)}$ ως το βάρος της διαδρομής $p_{i,j}^{(k)}$. Αν οι κορυφές v_i και v_j είναι διακεκριμένες ($i \neq j$) και υπάρχει κορυφή v_m τέτοια ώστε η ακμή (v_m, v_j) ανήκει στη διαδρομή $p_{i,j}^{(k)}$, τότε μπορούμε να διασπάσουμε τη διαδρομή $p_{i,j}^{(k)}$ σε μια διαδρομή από την κορυφή v_i προς την κορυφή v_m και την ακμή (v_m, v_j) . Επειδή η διαδρομή $p_{i,j}^{(k)}$ έχει το πολύ k ακμές, η διαδρομή από την κορυφή v_i προς την κορυφή v_m έχει το πολύ $(k - 1)$ ακμές. Από αυτό προκύπτει

ότι $p_{i,j}^{(k)} = p_{i,m}^{(k-1)} + (v_m, v_j)$. Συνεπώς το $d_{i,j}^{(k)}$ είναι το ελάχιστο του $d_{i,j}^{(k-1)}$ (το βάρος της συντομότερης διαδρομής από την κορυφή v_i στην κορυφή v_j) και το ελάχιστο κάθε διαδρομής από την κορυφή v_i προς την κορυφή v_j που περιέχει το πολύ k ακμές, που προκύπτει από την εξέταση όλων των πιθανών προγόνων v_m της κορυφής v_j . Έτσι, το $d_{i,j}^{(k)}$ καθορίζεται ως

$$d_{i,j}^{(k)} = \min \left\{ d_{i,j}^{(k-1)}, \min_{1 \leq m \leq n} \{d_{i,m}^{(k-1)} + w(v_m, v_j)\} \right\}.$$

Δεδομένου ότι $w(v_j, v_j) = 0$, η παραπάνω εξίσωση απλοποιείται σε

$$d_{i,j}^{(k)} = \min_{1 \leq m \leq n} \{d_{i,m}^{(k-1)} + w(v_m, v_j)\}. \quad (4)$$

όπου $d_{i,j}^{(1)} = a_{i,j}$. Έστω $D^{(k)} = (d_{i,j}^{(k)})$ ο πίνακας που αποθηκεύει τα βάρη των συντομότερων διαδρομών μεταξύ οποιωνδήποτε δύο κορυφών που περιέχουν το πολύ k ακμές. Εφόσον μία συντομότερη διαδρομή μεταξύ οποιουδήποτε ζεύγους κορυφών έχει το πολύ $(n-1)$ ακμές, ο πίνακας $D^{(n-1)}$ περιέχει τα βάρη των συντομότερων διαδρομών μεταξύ όλων των ζευγών κορυφών στο G .

Ο πίνακας $D^{(k)}$ υπολογίζεται από τον πίνακα $D^{(k-1)}$ χρησιμοποιώντας την Εξίσωση 4, και τον πίνακα γειτνίασης με βάρη A , χρησιμοποιώντας έναν τροποποιημένο αλγόριθμο πολλαπλασιασμού πινάκων. Ο τροποποιημένος αλγόριθμος χρησιμοποιεί πρόσθεση και ελαχιστοποίηση αντί για πολλαπλασιασμό και πρόσθεση. Συγκεκριμένα, όταν πολλαπλασιάζουμε $C = A \times B$ με τον τυπικό πολλαπλασιασμό πινάκων, έχουμε $c_{i,j} = \sum_{k=1}^{k=n} a_{i,k} \times b_{k,j}$, ενώ όταν χρησιμοποιούμε πρόσθεση και ελαχιστοποίηση παίρνουμε $c_{i,j} = \min_{k=1}^{k=n} (a_{i,k} + b_{k,j})$. Αυτός ο τροποποιημένος πολλαπλασιασμός πινάκων υπολογίζει τον $D^{(k)}$ πολλαπλασιάζοντας τους πίνακες $D^{(k-1)}$ και A για $2 \leq k \leq n-1$. Εφόσον $D^{(1)} = A$, προκύπτει ότι $D^{(k)} = A^k$ (δηλαδή η k -οστή δύναμη του A). Ο πίνακας $D^{(n-1)}$ υπολογίζεται υπολογίζοντας $A^2, A^4, A^8, \dots, A^{n-1}$. Αυτό απαιτεί μόνο $\lceil \log(n-1) \rceil$ τροποποιημένους πολλαπλασιασμούς. Το Σχήμα 11 απεικονίζει τα βήματα υπολογισμού του αλγορίθμου.

Χρησιμοποιώντας τον τυπικό αλγόριθμο για τον πολλαπλασιασμό πινάκων με πολυπλοκότητα $\Theta(n^3)$, η πολυπλοκότητα του αλγορίθμου για τις συντομότερες διαδρομές μεταξύ όλων των ζευγών είναι $\Theta(n^3 \log n)$. Δεδομένου ότι η πολυπλοκότητα του καλύτερα γνωστού σειριακού αλγορίθμου για τις συντομότερες διαδρομές μεταξύ όλων των ζευγών είναι $\Theta(n^3)$, αυτός ο αλγόριθμος δεν είναι βέλτιστος. Ωστόσο, ο αλγόριθμος παρουσιάζει υψηλό βαθμό παραλληλισμού.

Παράλληλη Υλοποίηση

Η παραλληλοποίηση του προηγούμενου αλγορίθμου είναι άμεση. Το κάθε ένα από τα $\lceil \log(n-1) \rceil$ βήματα του αλγορίθμου απαιτεί έναν πολλαπλασιασμό πινάκων. Για απλοποίηση της παρουσίασης, υποθέτουμε ότι $n-1$ είναι δύναμη του 2, αλλά η ανάλυση ισχύει για οποιαδήποτε τιμή του n .

Έστω ότι διαθέτουμε έναν παραλληλο υπολογιστή συνδεδεμένο ως υπερκύβο με n^3 επεξεργαστές. Ο πολλαπλασιασμός δύο $n \times n$ πινάκων απαιτεί χρόνο $\Theta(\log n)$ χρησιμοποιώντας τον αλγόριθμο DNS (Dekel, Nassimi, Sahni). Επομένως, ο χρόνος για την εκτέλεση $\log n$ τροποποιημένων πολλαπλασιασμών πινάκων είναι $\Theta(\log^2 n)$. Για το πρόβλημα των συντομότερων διαδρομών μεταξύ όλων των ζευγών, ο πιο γνωστός σειριακός αλγόριθμος έχει πολυπλοκότητα $\Theta(n^3)$. Έτσι, η επιτάχυνση και η αποδοτικότητα δίνονται ως εξής:

$$S = \frac{\Theta(n^3)}{\Theta(\log^2 n)}$$

$$E = \frac{1}{\Theta(\log^2 n)}$$



$$A^1 = \begin{pmatrix} 0 & 2 & 3 & \infty & \infty & \infty & \infty & \infty & \infty \\ \infty & 0 & \infty & \infty & \infty & 1 & \infty & \infty & \infty \\ \infty & \infty & 0 & 1 & 2 & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & 0 & \infty & \infty & 2 & \infty & \infty \\ \infty & \infty & \infty & \infty & 0 & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & \infty & 0 & 2 & 3 & 2 \\ \infty & \infty & \infty & \infty & 1 & \infty & 0 & 1 & \infty \\ \infty & \infty & \infty & \infty & \infty & \infty & \infty & 0 & \infty \\ \infty & \infty & \infty & \infty & \infty & \infty & \infty & 1 & 0 \end{pmatrix}$$

$$A^2 = \begin{pmatrix} 0 & 2 & 3 & 4 & 5 & 3 & \infty & \infty & \infty \\ \infty & 0 & \infty & \infty & \infty & 1 & 3 & 4 & 3 \\ \infty & \infty & 0 & 1 & 2 & \infty & 3 & \infty & \infty \\ \infty & \infty & \infty & 0 & 3 & \infty & 2 & 3 & \infty \\ \infty & \infty & \infty & \infty & 0 & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & 3 & 0 & 2 & 3 & 2 \\ \infty & \infty & \infty & \infty & \infty & \infty & 0 & 1 & \infty \\ \infty & \infty & \infty & \infty & \infty & \infty & \infty & 0 & \infty \\ \infty & \infty & \infty & \infty & \infty & \infty & \infty & 1 & 0 \end{pmatrix}$$

$$A^4 = \begin{pmatrix} 0 & 2 & 3 & 4 & 5 & 3 & 5 & 6 & 5 \\ \infty & 0 & \infty & \infty & 4 & 1 & 3 & 4 & 3 \\ \infty & \infty & 0 & 1 & 2 & \infty & 3 & 4 & \infty \\ \infty & \infty & \infty & 0 & 3 & \infty & 2 & 3 & \infty \\ \infty & \infty & \infty & \infty & 0 & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & 3 & 0 & 2 & 3 & 2 \\ \infty & \infty & \infty & \infty & 1 & \infty & 0 & 1 & \infty \\ \infty & \infty & \infty & \infty & \infty & \infty & \infty & 0 & \infty \\ \infty & \infty & \infty & \infty & \infty & \infty & \infty & 1 & 0 \end{pmatrix}$$

$$A^8 = \begin{pmatrix} 0 & 2 & 3 & 4 & 5 & 3 & 5 & 6 & 5 \\ \infty & 0 & \infty & \infty & 4 & 1 & 3 & 4 & 3 \\ \infty & \infty & 0 & 1 & 2 & \infty & 3 & 4 & \infty \\ \infty & \infty & \infty & 0 & 3 & \infty & 2 & 3 & \infty \\ \infty & \infty & \infty & \infty & 0 & \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty & 3 & 0 & 2 & 3 & 2 \\ \infty & \infty & \infty & \infty & 1 & \infty & 0 & 1 & \infty \\ \infty & \infty & \infty & \infty & \infty & \infty & \infty & 0 & \infty \\ \infty & \infty & \infty & \infty & \infty & \infty & \infty & 1 & 0 \end{pmatrix}$$

Σχήμα 11: Ένα παράδειγμα του αλγορίθμου για τις συντομότερες διαδρομές μεταξύ όλων των ζευγών.

Μπορούμε να βελτιώσουμε την αποδοτικότητα αυτής της υλοποίησης σε $\Theta(1/\log n)$ αν χρησιμοποιήσουμε $\Theta(n^3/\log n)$ επεξεργαστές στον αλγόριθμο πολλαπλασιασμού πινάκων DNS. Παρ' όλα αυτά, παραμένει μια μη κλιμακώσιμη παράλληλη υλοποίηση. Ωστόσο, πρόκειται για τον ταχύτερο γνωστό παράλληλο αλγόριθμο για την επίλυση αυτού του προβλήματος. Στις επόμενες ενότητες παρουσιάζουμε μερικούς παράλληλους αλγορίθμους που δεν είναι τόσο γρήγοροι, αλλά είναι βέλτιστοι ως προς το κόστος.

4.2 Ο αλγόριθμος του Dijkstra

Στην Ενότητα 3 παρουσιάσαμε τον αλγόριθμο του Dijkstra για την εύρεση των συντομότερων διαδρομών από μια κορυφή v προς όλες τις άλλες κορυφές ενός γραφήματος. Ο

αλγόριθμος αυτός μπορεί επίσης να χρησιμοποιηθεί για την επίλυση του προβλήματος των συντομότερων διαδρομών για όλα τα ζεύγη εκτελώντας τον αλγόριθμο μια μόνο πηγής για κάθε επεξεργαστή, για κάθε κορυφή v . Αναφερόμαστε σε αυτόν τον αλγόριθμο ως τον αλγόριθμο του Dijkstra των συντομότερων διαδρομών για όλα τα ζεύγη. Από την στιγμή που η πολυπλοκότητα του αλγορίθμου του Dijkstra για μονή πηγή είναι $\Theta(n^2)$, η πολυπλοκότητα του αλγορίθμου για όλα τα ζεύγη είναι $\Theta(n^3)$.

Παράλληλες Υλοποιήσεις

Το πρόβλημα των συντομότερων διαδρομών για όλα τα ζεύγη του Dijkstra μπορεί να παραλληλοποιηθεί με δύο διακριτούς τρόπους. Μια προσέγγιση διαχωρίζει τις κορυφές με-ταξύ διαφορετικών επεξεργαστών και αναθέτει σε κάθε επεξεργαστή τον υπολογισμό των συντομότερων διαδρομών μονής πηγής για όλες τις κορυφές που του έχουν ανατεθεί. Αναφερόμαστε σε αυτή την προσέγγιση ως την **υλοποίηση με διαμέριση κατά πηγή** (source-partitioned formulation). Μια άλλη προσέγγιση αναθέτει κάθε κορυφή σε ένα σύνολο επεξεργαστών και χρησιμοποιεί την παράλληλη υλοποίηση του αλγορίθμου μονής πηγής (Ενότητα 3) για να λύσει το πρόβλημα στο κάθε σύνολο επεξεργαστών. Αναφερόμαστε σε αυτή την προσέγγιση ως **παράλληλη υλοποίηση πηγής** (source-parallel formulation). Οι επόμενες ενότητες συζητούν και αναλύουν αυτές τις δύο προσεγγίσεις.

Υλοποίηση με διαμέριση κατά πηγή Η παράλληλη υλοποίηση με διαμέριση κατά πηγή του αλγορίθμου του Dijkstra χρησιμοποιεί n επεξεργαστές. Κάθε επεξεργαστής P_i βρίσκει τις συντομότερες διαδρομές από την κορυφή v_i προς όλες τις υπόλοιπες κορυφές εκτελώντας τον σειριακό αλγόριθμο των συντομότερων διαδρομών μονής πηγής του Dijkstra. Δεν απαιτείται επικοινωνία μεταξύ των επεξεργαστών. Συνεπώς, ο παράλληλος χρόνος εκτέλεσης αυτής της υλοποίησης είναι

$$T_P = \Theta(n^2).$$

Δεδομένου ότι ο σειριακός χρόνος εκτέλεσης είναι $W = \Theta(n^3)$, η επιτάχυνση και η αποδοτικότητα προκύπτουν ως εξής:

$$S = \frac{\Theta(n^3)}{\Theta(n^2)}$$

$$E = \Theta(1) \tag{5}$$

Θα μπορούσε να φαίνεται ότι, λόγω της απουσίας επικοινωνίας, αυτή είναι μια εξαιρετική παράλληλη υλοποίηση. Ωστόσο, αυτό δεν ισχύει πλήρως. Ο αλγόριθμος μπορεί να χρησιμοποιήσει το πολύ n επεξεργαστές. Συνεπώς, η συνάρτηση ισοαποτελεσματικότητας λόγω παράλληλης εκτέλεσης είναι $\Theta(p^3)$, που είναι η συνολική ισοαποτελεσματικότητα του αλγορίθμου. Αν ο αριθμός των διαθέσιμων επεξεργαστών για την επίλυση του προβλήματος είναι μικρός (δηλαδή, $n = \Theta(p)$), τότε αυτός ο αλγόριθμος έχει καλή απόδοση. Όμως, εάν ο αριθμός των επεξεργαστών είναι μεγαλύτερος από n , άλλοι αλγόριθμοι θα υπερέχουν τελικά αυτού του αλγορίθμου λόγω της φτωχής κλιμάκωσής του.

Παράλληλη υλοποίηση πηγής Το κύριο πρόβλημα της παράλληλης υλοποίησης με διαμέριση κατά πηγή είναι ότι μπορεί να κρατήσει απασχολημένους σε ωφέλιμο έργο το πολύ n επεξεργαστές. Η απόδοση μπορεί να βελτιωθεί εάν χρησιμοποιηθεί η παράλληλη υλοποίηση του αλγορίθμου μονής πηγής του Dijkstra (Ενότητα 3) για την επίλυση του προβλήματος

για κάθε κορυφή v . Η παράλληλη υλοποίηση πηγής είναι παρόμοια με την υλοποίηση με διαμέριση κατά πηγή, εκτός του ότι ο αλγόριθμος μονής πηγής εκτελείται σε ξένα υποσύνολα επεξεργαστών.

Συγκεκριμένα, οι p επεξεργαστές χωρίζονται σε n διαμερίσεις, κάθε μια με p/n επεξεργαστές (αυτή η υλοποίηση έχει ενδιαφέρον μόνο αν $p > n$). Κάθε ένα από τα n προβλήματα των ελαχίστων διαδρομών μονής πηγής επιλύεται από μία από τις n διαμερίσεις. Με άλλα λόγια, πρώτα παραλληλοποιούμε το πρόβλημα των συντομότερων διαδρομών για όλα τα ζεύγη εκχωρώντας κάθε κορυφή σε ξεχωριστό σύνολο επεξεργαστών και στη συνέχεια παραλληλοποιώντας τον αλγόριθμο μονής πηγής χρησιμοποιώντας το σύνολο των p/n επεξεργαστών για την επίλυσή του. Ο συνολικός αριθμός επεξεργαστών που μπορεί να χρησιμοποιηθεί αποδοτικά με αυτή τη υλοποίηση είναι $O(n^2)$.



Σχήμα 12: Διαμέριση ενός πλέγματος $\sqrt{p} \times \sqrt{p}$ σε n υποπλέγματα, καθένα με μέγεθος $\sqrt{p/n} \times \sqrt{p/n}$. Στο παράδειγμα αυτό, $p = 16$ και $n = 4$. Κάθε ένα από τα πλέγματα μεγέθους $\sqrt{p/n} \times \sqrt{p/n} = 2 \times 2$ επιλύει το πρόβλημα των συντομότερων διαδρομών μονής πηγής για μια δοσμένη κορυφή πηγής. Στο παράδειγμα αυτό, κάθε υποπλέγμα επισημαίνεται από την κορυφή πηγής του αντίστοιχου αλγορίθμου μονής πηγής.

Η ανάλυση που παρουσιάστηκε στην Ενότητα 3 μπορεί να χρησιμοποιηθεί για να αντληθεί η απόδοση αυτής της υλοποίησης του αλγορίθμου του Dijkstra για όλα τα ζεύγη. Έστω ότι έχουμε ένα πλέγμα p -επεξεργαστών τέτοιο ώστε το $\sqrt{p}$ να είναι πολλαπλάσιο του $\sqrt{n}$. Το πλέγμα $\sqrt{p} \times \sqrt{p}$ χωρίζεται σε n υποπλέγματα, καθένα με μέγεθος $\sqrt{p/n} \times \sqrt{p/n}$. Το Σχήμα 12 απεικονίζει αυτή την διαμέριση. Αν ο αλγόριθμος μονής πηγής εκτελεστεί σε κάθε υποπλέγμα, ο παράλληλος χρόνος εκτέλεσης είναι

$$T_P = \overbrace{\Theta\left(\frac{n^3}{p}\right)}^{\text{υπολογισμός}} + \overbrace{\Theta(\sqrt{np})}^{\text{επικοινωνία}} \quad (6)$$

Παρατηρήστε τις ομοιότητες μεταξύ της Εξίσωσης (6) και της (2). Οι ομοιότητες αυτές δεν προκαλούν εντύπωση, αφού κάθε σύνολο των p/n επεξεργαστών σχηματίζει ένα πλέγμα και εκτελεί τον υπολογισμό ανεξάρτητα. Επομένως, ο χρόνος που απαιτείται από κάθε σύνολο p/n επεξεργαστών για να λύσει το πρόβλημα μονής πηγής καθορίζει τον συνολικό χρόνο εκτέλεσης. Δεδομένου ότι ο σειριακός χρόνος εκτέλεσης είναι $W = \Theta(n^3)$, η επιτάχυνση και η αποδοτικότητα είναι οι εξής:

$$S = \frac{\Theta(n^3)}{\Theta(n^3/p) + \Theta(\sqrt{np})}$$

$$E = \frac{1}{1 + \Theta(p^{1.5}/n^{2.5})} \quad (7)$$

Από την Εξίσωση (7) βλέπουμε ότι για μια βέλτιστη ως προς το κόστος υλοποίηση $p^{1.5}/n^{2.5} = O(1)$. Επομένως, η υλοποίηση μπορεί να χρησιμοποιήσει έως και $O(n^{1.66})$ επεξεργαστές αποδοτικά. Η Εξίσωση (7) δείχνει επίσης ότι η συνάρτηση ισοαποδοτικότητας λόγω επικοινωνίας είναι $\Theta(p^{1.8})$. Η συνάρτηση ισοαποδοτικότητας λόγω συγχρονισμού είναι $\Theta(p^{1.5})$. Συνεπώς, η συνολική συνάρτηση ισοαποδοτικότητας είναι $\Theta(p^{1.8})$.

Συγκρίνοντας τις δύο παράλληλες υλοποιήσεις του αλγορίθμου όλων-των-ζευγών του Dijkstra, βλέπουμε ότι η υλοποίηση με διαμέριση κατά πηγή δεν έχει επικοινωνία, μπορεί να χρησιμοποιήσει το πολύ n επεξεργαστές, και λύνει το πρόβλημα σε χρόνο $\Theta(n^2)$. Αντίθετα, η παράλληλη υλοποίηση πηγής χρησιμοποιεί έως $n^{1.66}$ επεξεργαστές, έχει κάποιο κόστος επικοινωνίας, και λύνει το πρόβλημα σε χρόνο $\Theta(n^{1.33})$ όταν χρησιμοποιούνται $n^{1.66}$ επεξεργαστές. Έτσι, η παράλληλη υλοποίηση πηγής εκμεταλλεύεται περισσότερο τον παραλληλισμό από ότι η υλοποίηση με διαμέριση κατά πηγή.

4.3 Ο αλγόριθμος του Floyd

Ο αλγόριθμος του Floyd για την επίλυση του προβλήματος των συντομότερων διαδρομών για όλα τα ζεύγη βασίζεται στην εξής παρατήρηση. Έστω $G = (V, E, w)$ το γράφημα με βάρη και έστω $V = \{v_1, v_2, \dots, v_n\}$ οι κορυφές του G . Θεωρήστε ένα υποσύνολο $\{v_1, v_2, \dots, v_k\}$ κορυφών για κάποιο k με $k \leq n$. Για κάθε ζεύγος κορυφών $v_i, v_j \in V$, θεωρήστε όλες τις διαδρομές από v_i προς v_j των οποίων οι ενδιάμεσες κορυφές ανήκουν στο σύνολο $\{v_1, v_2, \dots, v_k\}$. Έστω $p_{i,j}^{(k)}$ η διαδρομή ελάχιστου βάρους μεταξύ αυτών, και $d_{i,j}^{(k)}$ το βάρος της $p_{i,j}^{(k)}$. Εάν η κορυφή v_k δεν ανήκει στη συντομότερη διαδρομή από v_i προς v_j , τότε η $p_{i,j}^{(k)}$ ταυτίζεται με την $p_{i,j}^{(k-1)}$. Εάν όμως v_k είναι στην $p_{i,j}^{(k)}$, μπορούμε να διασπάσουμε την $p_{i,j}^{(k)}$ σε δύο διαδρομές, μία από v_i προς v_k και μία από v_k προς v_j . Κάθε μία από αυτές τις διαδρομές χρησιμοποιεί κορυφές από το σύνολο $\{v_1, v_2, \dots, v_{k-1}\}$. Έτσι, $d_{i,j}^{(k)} = d_{i,k}^{(k-1)} + d_{k,j}^{(k-1)}$. Αυτές οι παρατηρήσεις εκφράζονται στην ακόλουθη εξίσωση αναδρομής:

$$d_{i,j}^{(k)} = \begin{cases} w(v_i, v_j), & \text{εάν } k = 0, \\ \min\{d_{i,j}^{(k-1)}, d_{i,k}^{(k-1)} + d_{k,j}^{(k-1)}\}, & \text{εάν } k \geq 1. \end{cases} \quad (8)$$

Το μήκος του συντομότερου μονοπατιού από v_i προς v_j είναι $d_{i,j}^{(n)}$. Γενικά, η λύση είναι ένας πίνακας $D^{(n)} = d_{i,j}^{(n)}$.

Ο αλγόριθμος του Floyd επιλύει την Εξίσωση 8 με προσέγγιση από κάτω προς τα πάνω (bottom-up) κατά σειρά αυξανόμενων τιμών του k . Ο αλγόριθμος 5 παρουσιάζει τον αλγόριθμο όλων-των-ζευγών του Floyd. Ο χρόνος εκτέλεσης του αλγορίθμου καθορίζεται από τους τριπλά εμφωλευμένους βρόχους for στις γραμμές 4–7. Κάθε εκτέλεση της γραμμής 7 απαιτεί $\Theta(1)$ χρόνο. Συνεπώς, η πολυπλοκότητα του αλγορίθμου είναι $\Theta(n^3)$. Ο Αλγόριθμος 5 φαίνεται να υπονοεί ότι πρέπει να αποθηκεύσουμε n πίνακες μεγέθους $n \times n$. Ωστόσο, κατά τον υπολογισμό του πίνακα $D^{(k)}$, απαιτείται μόνο ο πίνακας $D^{(k-1)}$. Συνεπώς, πρέπει να αποθηκευτούν το πολύ δύο πίνακες $n \times n$. Οπότε, η συνολική πολυπλοκότητα μνήμης είναι $\Theta(n^2)$. Επιπλέον, ο αλγόριθμος λειτουργεί σωστά ακόμη και όταν χρησιμοποιείται μόνο ένα αντίγραφο του D .

```

1: procedure FLOYD_ALL_PAIRS_SP( $A$ )
2:    $D^{(0)} \leftarrow A$ 
3:   for  $k = 1$  to  $n$  do
4:     for  $i = 1$  to  $n$  do
5:       for  $j = 1$  to  $n$  do
6:          $d_{i,j}^{(k)} \leftarrow \min\{d_{i,j}^{(k-1)}, d_{i,k}^{(k-1)} + d_{k,j}^{(k-1)}\}$ 
7:       end for
8:     end for
9:   end for
10:  return  $D^{(n)}$ 
11: end procedure

```

Αλγόριθμος 5: Ο αλγόριθμος του Floyd για τις συντομότερες διαδρομές για όλα τα ζεύγη. Αυτός ο αλγόριθμος υπολογίζει τον γράφο $G = (V, E)$ των συντομότερων διαδρομών μεταξύ όλων των ζευγών με τον πίνακα γειτνίασης A .

Παράλληλη Υλοποίηση

Μια γενική παράλληλη υλοποίηση του αλγορίθμου του Floyd αναθέτει το έργο του υπολογισμού του πίνακα $D^{(k)}$ για κάθε τιμή του k σε ένα σύνολο επεξεργαστών. Έστω p ο αριθμός των διαθέσιμων επεξεργαστών. Ο πίνακας $D^{(k)}$ διαμερίζεται σε p μέρη, και κάθε μέρος ανατίθεται σε έναν επεξεργαστή. Κάθε επεξεργαστής υπολογίζει τις τιμές $D^{(k)}$ της διαμέρισής του. Για να το επιτύχει αυτό, ένας επεξεργαστής πρέπει να έχει πρόσβαση στα αντίστοιχα τμήματα της k -οστής γραμμής και της k -οστής στήλης του πίνακα $D^{(k-1)}$. Η ακόλουθη ενότητα περιγράφει μία τεχνική για τη διαμέριση του πίνακα $D^{(k)}$.

Απεικόνιση Μπλοκ Σκακιέρας (Block-checkerboard mapping)

Μια μέθοδος για τη διαμέριση του πίνακα $D^{(k)}$ είναι η χρήση της απεικόνισης μπλοκ σκακιέρας. Συγκεκριμένα, ο πίνακας $D^{(k)}$ διαιρείται σε p τετράγωνα μεγέθους $(n/\sqrt{p}) \times (n/\sqrt{p})$, και κάθε τετράγωνο ανατίθεται σε έναν από τους p επεξεργαστές. Είναι χρήσιμο να θεωρήσουμε τους p επεξεργαστές διατεταγμένους σε ένα λογικό πλέγμα μεγέθους $\sqrt{p} \times \sqrt{p}$. Σημειώστε ότι πρόκειται μόνο για εννοιολογική διάταξη και δεν αντανακλά απαραίτητα το πραγματικό δίκτυο διασύνδεσης των επεξεργαστών. Αναφερόμαστε στον επεξεργαστή που βρίσκεται στην i -οστή γραμμή και j -οστή στήλη ως $P_{i,j}$. Στον επεξεργαστή $P_{i,j}$ ανατίθεται ένα τετράγωνο υπομπλοκ του $D^{(k)}$ του οποίου η επάνω αριστερή γωνία είναι $((i-1)n/\sqrt{p} + 1, (j-1)n/\sqrt{p} + 1)$ και η κάτω δεξιά γωνία του είναι $(in/\sqrt{p}, jn/\sqrt{p})$. Κάθε επεξεργαστής ενημερώνει το τμήμα του πίνακα κατά τη διάρκεια κάθε επανάληψης. Το Σχήμα 13 απεικονίζει την τεχνική διαμέρισης τύπου σκακιέρας (block-checkerboard partitioning).



Σχήμα 13: (α) Ο πίνακας $D^{(k)}$ διαμερισμένος με αντιστοίχιση σε μπλοκ σκακιέρας σε $\sqrt{p} \times \sqrt{p}$ υπομπλοκ, και (β) το τετραγωνικό υπομπλοκ του $D^{(k)}$ που ανατίθεται στον επεξεργαστή $P_{i,j}$.



Σχήμα 14: (α) Μοτίβα επικοινωνίας που χρησιμοποιούνται στην αντιστοίχιση σε μπλοκ σκακιέρας. Κατά τον υπολογισμό του $d_{i,j}^{(k)}$, πληροφορίες πρέπει να αποσταλούν στον επιλεγμένο επεξεργαστή από δύο άλλους επεξεργαστές κατά μήκος της ίδιας γραμμής και στήλης. (β) Η γραμμή και η στήλη από $\sqrt{p}$ επεξεργαστές που περιέχουν την k -οστή γραμμή και στήλη, τις αποστέλλουν κατά μήκος των στηλών και γραμμών επεξεργαστών.

Κατά την k -οστή επανάληψη του αλγορίθμου, κάθε επεξεργαστής $P_{i,j}$ χρειάζεται συγκεκριμένα τμήματα της k -οστής γραμμής και της k -οστής στήλης του πίνακα $D^{(k-1)}$. Για παράδειγμα, για να υπολογίσει το $d_{i,r}^{(k)}$, πρέπει να λάβει τα $d_{i,k}^{(k-1)}$ και $d_{k,r}^{(k-1)}$. Όπως φαίνεται στο Σχήμα 14, το $d_{i,k}^{(k-1)}$ βρίσκεται σε έναν επεξεργαστή κατά μήκος της ίδιας γραμμής, ενώ το στοιχείο $d_{k,r}^{(k-1)}$ βρίσκεται σε έναν επεξεργαστή κατά μήκος της ίδιας στήλης με τον $P_{i,j}$. Τα τμήματα μεταφέρονται ως εξής. Κατά την k -οστή επανάληψη του αλγορίθμου, καθένας από τους $\sqrt{p}$ επεξεργαστές που περιέχουν μέρος της k -οστής γραμμής, το αποστέλλει στους $\sqrt{p} - 1$ επεξεργαστές της ίδιας στήλης. Αντίστοιχα, καθένας από τους $\sqrt{p}$ επεξεργαστές που περιέχουν μέρος της k -οστής στήλης, το αποστέλλει στους $\sqrt{p} - 1$ επεξεργαστές της ίδιας γραμμής.

Ο Αλγόριθμος 6 παρουσιάζει την παράλληλη υλοποίηση του αλγορίθμου του Floyd χρησιμοποιώντας κατανομή τύπου μπλοκ σκακιέρας. Αναλύουμε την απόδοση αυτού του αλγορίθμου σε έναν υπερκύβο με p επεξεργαστές και δρομολόγηση άμεσης προώθησης. Ένα εικονικό πλέγμα $\sqrt{p} \times \sqrt{p}$ μπορεί να αντιστοιχιστεί σε έναν υπερκύβο p επεξεργαστών. Θυμηθείτε ότι κάθε γραμμή και κάθε στήλη αυτού του εικονικού πλέγματος είναι ένας υπερκύβος από $\sqrt{p}$ επεξεργαστές. Κατά τη διάρκεια κάθε επανάληψης του αλγορίθμου, η k -οστή γραμμή και η k -οστή στήλη των επεξεργαστών εκτελούν μια εκπομπή ενός-προς-όλους κατά μήκος μιας γραμμής ή στήλης $\sqrt{p}$ επεξεργαστών. Κάθε τέτοιος επεξεργαστής έχει $n/\sqrt{p}$ στοιχεία της k -οστής γραμμής ή στήλης, οπότε στέλνει $n/\sqrt{p}$ στοιχεία. Αυτή η εκπομπή απαιτεί χρόνο $\Theta((n \log p)/\sqrt{p})$. Το βήμα συγχρονισμού στη γραμμή 7 απαιτεί χρόνο $\Theta(\log p)$. Δεδομένου ότι κάθε επεξεργαστής έχει ανατεθεί n^2/p στοιχεία του πίνακα $D^{(k)}$, ο χρόνος για τον υπολογισμό των αντίστοιχων τιμών $D^{(k)}$ είναι $\Theta(n^2/p)$.

```

1: procedure FLOYD_CHECKERBOARD( $D^{(0)}$ )
2:   for  $k = 1$  to  $n$  do
3:     for all processors  $P_{i,j}$  that have a segment of the  $k$ -th row of  $D^{(k-1)}$  do
4:       broadcast it to the processors  $P_{*,j}$ 
5:     end for
6:     for all processors  $P_{i,j}$  that have a segment of the  $k$ -th column of  $D^{(k-1)}$  do
7:       broadcast it to the processors  $P_{i,*}$ 
8:     end for
9:     for all processors  $P_{i,j}$  do
10:      wait to receive the needed segments
11:    end for
12:    for all processors  $P_{i,j}$  do
13:      compute its part of the  $D^{(k)}$  matrix
14:    end for
15:  end for
16: end procedure

```

Αλγόριθμος 6: Παράλληλη υλοποίηση του αλγορίθμου του Floyd με κατανομή τύπου μπλοκ σκακιέρας (block-checkerboard partitioning). Το $P_{*,j}$ δηλώνει όλους τους επεξεργαστές της j -οστής στήλης και το $P_{i,*}$ όλους τους επεξεργαστές της i -οστής γραμμής. Ο πίνακας $D^{(0)}$ είναι ο πίνακας γειτνίασης.

Συνεπώς, ο παράλληλος χρόνος εκτέλεσης της υλοποίησης τύπου μπλοκ σκακιέρας του αλγορίθμου του Floyd είναι:

$$T_P = \overbrace{\Theta\left(\frac{n^3}{p}\right)}^{\text{υπολογισμός}} + \overbrace{\Theta\left(\frac{n^2}{\sqrt{p}} \log p\right)}^{\text{επικοινωνία}}$$

Δεδομένου ότι ο σειριακός χρόνος εκτέλεσης είναι: $W = \Theta(n^3)$, η επιτάχυνση και η απόδοση είναι οι ακόλουθες:

$$S = \frac{\Theta(n^3)}{\Theta(n^3/p) + \Theta((n^2 \log p)/\sqrt{p})}$$

$$E = \frac{1}{1 + \Theta\left(\frac{\sqrt{p} \log p}{n}\right)} \quad (9)$$

Από την Εξίσωση 9 βλέπουμε ότι για μια υλοποίηση βέλτιστου κόστους ισχύει: $(\sqrt{p} \log p)/n = O(1)$. Επομένως, η μέθοδος κατά μπλοκ σκακιέρας μπορεί να χρησιμοποιήσει αποδοτικά έως και $O(n^2/\log^2 n)$ επεξεργαστές υπερκύβου. Η Εξίσωση 9 μπορεί επίσης να χρησιμοποιηθεί για την εξαγωγή της συνάρτησης ισοαποδοτικότητας λόγω επικοινωνίας, η οποία είναι $\Theta(p^{1.5} \log^3 p)$. Η συνάρτηση ισοαποδοτικότητας λόγω συγχρονισμού είναι $\Theta(p^{1.5})$. Συνεπώς, η συνολική συνάρτηση ισοαποδοτικότητας είναι $\Theta(p^{1.5} \log^3 p)$. Ο παράλληλος χρόνος εκτέλεσης, η επιτάχυνση και η συνάρτηση ισοαποδοτικότητας για ένα πλέγμα μπορούν να προσδιοριστούν με παρόμοιο τρόπο (Πίνακας 6).

Επιτάχυνση των Πραγμάτων

Στη υλοποίηση κατά μπλοκ σκακιέρας του αλγορίθμου του Floyd, ένα βήμα συγχρονισμού διασφαλίζει ότι όλοι οι επεξεργαστές διαθέτουν τα κατάλληλα τμήματα του πίνακα $D^{(k-1)}$ πριν υπολογίσουν τα στοιχεία του πίνακα $D^{(k)}$ (γραμμή 7 στον Αλγόριθμο 6). Με άλλα λόγια, η k -οστή επανάληψη ξεκινά μόνο όταν η $(k-1)$ -οστή επανάληψη έχει ολοκληρωθεί και τα σχετικά τμήματα του πίνακα $D^{(k-1)}$ έχουν μεταδοθεί σε όλους τους επεξεργαστές. Το βήμα συγχρονισμού μπορεί να αφαιρεθεί χωρίς να επηρεάσει την ορθότητα του αλγορίθμου. Για να επιτευχθεί αυτό, ένας επεξεργαστής αρχίζει να εργάζεται στην k -οστή επανάληψη μόλις έχει υπολογίσει την $(k-1)$ -οστή επανάληψη και διαθέτει τα σχετικά τμήματα του πίνακα $D^{(k-1)}$. Αυτή η υλοποίηση ονομάζεται **μπλοκ σκακιέρας με σωλήνωση** (pipelined block checkerboarding).

Θεωρούμε ένα πλέγμα p επεξεργαστών. Υποθέτουμε ότι ο επεξεργαστής $P_{i,j}$ αρχίζει να εργάζεται στην k -οστή επανάληψη μόλις ολοκληρώσει την $(k-1)$ -οστή επανάληψη και λάβει τα σχετικά τμήματα του πίνακα $D^{(k-1)}$. Όταν ο επεξεργαστής $P_{i,j}$ διαθέτει στοιχεία της k -οστής γραμμής και έχει ολοκληρώσει την $(k-1)$ -οστή επανάληψη, αποστέλλει το τμήμα του πίνακα $D^{(k-1)}$ που είναι αποθηκευμένο τοπικά στους επεξεργαστές $P_{i,j-1}$ και $P_{i,j+1}$. Το κάνει αυτό διότι εκείνο το τμήμα του πίνακα $D^{(k-1)}$ χρησιμοποιείται για τον υπολογισμό του πίνακα $D^{(k)}$. Ομοίως, όταν ο επεξεργαστής $P_{i,j}$ διαθέτει στοιχεία της k -οστής στήλης και έχει ολοκληρώσει την $(k-1)$ -οστή επανάληψη, αποστέλλει το τμήμα του πίνακα $D^{(k-1)}$ που είναι αποθηκευμένο τοπικά στους επεξεργαστές $P_{i-1,j}$ και $P_{i+1,j}$. Όταν ο επεξεργαστής $P_{i,j}$ λαμβάνει στοιχεία του πίνακα $D^{(k)}$ από έναν επεξεργαστή της ίδιας γραμμής στο λογικό πλέγμα, τα αποθηκεύει τοπικά και τα προωθεί στον επεξεργαστή που βρίσκεται στην αντίθετη πλευρά από αυτή που τα έλαβε. Οι στήλες ακολουθούν ένα παρόμοιο πρωτόκολλο επικοινωνίας. Στοιχεία του πίνακα $D^{(k)}$ δεν προωθούνται όταν φτάνουν στα όρια

του πλέγματος. Το Σχήμα 15 απεικονίζει αυτό το πρωτόκολλο επικοινωνίας και τερματισμού για επεξεργαστές εντός μιας γραμμής (ή μιας στήλης).



Σχήμα 15: Πρωτόκολλο επικοινωνίας που ακολουθείται στη υλοποίηση σωλήνωσης κατά μπλοκ σκακιέρας του αλγορίθμου του Floyd. Υποθέτουμε ότι ο επεξεργαστής 4 τη χρονική στιγμή t έχει μόλις υπολογίσει ένα τμήμα της k -οστής στήλης του πίνακα $D^{(k-1)}$. Το αποστέλλει στους επεξεργαστές 3 και 5. Αυτοί οι επεξεργαστές λαμβάνουν το τμήμα τη χρονική στιγμή $t+1$ (όπου η μονάδα χρόνου είναι ο χρόνος που χρειάζεται ένα τμήμα πίνακα για να διανύσει τη ζεύξη επικοινωνίας μεταξύ γειτονικών επεξεργαστών). Ομοίως, οι επεξεργαστές που απέχουν περισσότερο από τον επεξεργαστή 4 λαμβάνουν το τμήμα αργότερα. Ο επεξεργαστής 1 (στο όριο του πλέγματος) δεν προωθεί το τμήμα μετά την παραλαβή του.

Θεωρούμε την κίνηση των τιμών στην πρώτη επανάληψη. Σε κάθε βήμα, $n/\sqrt{p}$ στοιχεία της πρώτης γραμμής αποστέλλονται από τον επεξεργαστή $P_{i,j}$ στον $P_{i+1,j}$. Ομοίως, στοιχεία της πρώτης στήλης αποστέλλονται από τον επεξεργαστή $P_{i,j}$ στον $P_{i,j+1}$. Κάθε τέτοιο βήμα απαιτεί χρόνο $\Theta(n/\sqrt{p})$. Μετά από $\Theta(\sqrt{p})$ βήματα, ο επεξεργαστής $P_{\sqrt{p},\sqrt{p}}$ λαμβάνει τα σχετικά στοιχεία της πρώτης γραμμής και της πρώτης στήλης σε χρόνο $\Theta(n)$. Οι τιμές των διαδοχικών γραμμών και στηλών ακολουθούν μετά από χρόνο $\Theta(n^2/p)$ σε κατάσταση σωλήνωσης. Επομένως, ο επεξεργαστής $P_{\sqrt{p},\sqrt{p}}$ ολοκληρώνει το μερίδιό του στον υπολογισμό του συντομότερου μονοπατιού σε χρόνο $\Theta(n^3/p) + \Theta(n)$. Όταν ο επεξεργαστής $P_{\sqrt{p},\sqrt{p}}$ έχει ολοκληρώσει την $(n-1)$ -οστή επανάληψη, αποστέλλει τις σχετικές τιμές της n -οστής γραμμής και στήλης στους υπόλοιπους επεξεργαστές. Αυτές οι τιμές φτάνουν στον επεξεργαστή $P_{1,1}$ σε χρόνο $\Theta(n)$. Ο συνολικός παράλληλος χρόνος εκτέλεσης αυτής της υλοποίησης είναι

$$T_P = \overbrace{\Theta\left(\frac{n^3}{p}\right)}^{\text{υπολογισμός}} + \overbrace{\Theta(n)}^{\text{επικοινωνία}}$$

Εφόσον ο σειριακός χρόνος εκτέλεσης είναι $W = \Theta(n^3)$, η επιτάχυνση και η αποδοτικότητα έχουν ως εξής:

$$S = \frac{\Theta(n^3)}{\Theta(n^3/p) + \Theta(n)}$$

$$E = \frac{1}{1 + \Theta(p/n^2)} \quad (10)$$

Από την Εξίσωση 10 βλέπουμε ότι για μια υλοποίηση βέλτιστου κόστους ισχύει $p/n^2 = O(1)$. Επομένως, η υλοποίηση σωλήνωσης του αλγορίθμου του Floyd χρησιμοποιεί αποδοτικά έως $O(n^2)$ επεξεργαστές. Επίσης, από την Εξίσωση 10 μπορούμε να εξάγουμε τη συνάρτηση ισοαποδοτικότητας λόγω επικοινωνίας, η οποία είναι $\Theta(p^{1.5})$. Αυτή είναι και η συνολική συνάρτηση ισοαποδοτικότητας. Συγκρίνοντας την υλοποίηση σωλήνωσης με τη συγχρονισμένη υλοποίηση κατά μπλοκ σκακιέρας, διαπιστώνουμε ότι η πρώτη είναι σημαντικά ταχύτερη.

Πίνακας 6: Η απόδοση και η κλιμακωσιμότητα των αλγορίθμων συντομότερων μονοπατιών όλων των ζευγών σε διάφορες αρχιτεκτονικές.

	Μέγιστος Αριθμός Επεξεργαστών για $E = \Theta(1)$	Αντίστοιχος Παράλληλος Χρόνος	Συνάρτηση Ισοαποδοτικότητας
Διαμερισμένη πηγή του Dijkstra			
Υπερκύβος	$\Theta(n)$	$\Theta(n^2)$	$\Theta(p^3)$
Πλέγμα	$\Theta(n)$	$\Theta(n^2)$	$\Theta(p^3)$
Παραλληλοποιημένη πηγή του Dijkstra			
Υπερκύβος	$\Theta(n^2/\log n)$	$\Theta(n \log n)$	$\Theta((p \log p)^{1.5})$
Πλέγμα	$\Theta(n^{1.67})$	$\Theta(n^{1.33})$	$\Theta(p^{1.8})$
Απεικόνιση με μπλοκ σε λωρίδες του Floyd			
Υπερκύβος	$\Theta(n/\log n)$	$\Theta(n^2 \log n)$	$\Theta((p \log p)^3)$
Πλέγμα (SF)	$\Theta(n^{0.22})$	$\Theta(n^{2.78})$	$\Theta(p^{4.5})$
Πλέγμα (CT)	$\Theta(n/\log n)$	$\Theta(n^2 \log n)$	$\Theta((p \log p)^3)$
Απεικόνιση με μπλοκ σκακιέρας του Floyd			
Υπερκύβος	$\Theta(n^2/\log^2 n)$	$\Theta(n \log^2 n)$	$\Theta(p^{1.5} \log^3 p)$
Πλέγμα (SF)	$\Theta(n)$	$\Theta(n^2)$	$\Theta(p^3)$
Πλέγμα (CT)	$\Theta(n^{1.33})$	$\Theta(n^{1.67})$	$\Theta(p^{2.25})$
Σωλήνωση με μπλοκ σκακιέρας του Floyd			
Υπερκύβος	$\Theta(n^2)$	$\Theta(n)$	$\Theta(p^{1.5})$
Πλέγμα	$\Theta(n^2)$	$\Theta(n)$	$\Theta(p^{1.5})$

4.4 Συγκρίσεις Απόδοσης

Η απόδοση των αλγορίθμων συντομότερων μονοπατιών όλων των ζευγών που παρουσιάστηκαν προηγουμένως συνοψίζεται στον Πίνακα 6 για διάφορες αρχιτεκτονικές. Η υλοποίηση σωλήνωσης του Floyd είναι η πιο κλιμακώσιμη και μπορεί να χρησιμοποιήσει έως $\Theta(n^2)$

επεξεργαστές για να επιλύσει το πρόβλημα σε χρόνο $\Theta(n)$. Αυτή η παράλληλη υλοποίηση αποδίδει εξίσου καλά σε υπολογιστή με σύνδεση πλέγματος ή υπερκύβου. Επιπλέον, η απόδοσή της είναι ανεξάρτητη από τον τύπο της δρομολόγησης (αποθήκευση και προώθηση ή άμεση προώθηση).

5 Μεταβατική Κλειστότητα

Σε πολλές εφαρμογές επιθυμούμε να προσδιορίσουμε αν δύο οποιεσδήποτε κορυφές σε ένα γράφημα είναι συνδεδεμένες. Αυτό συνήθως γίνεται με την εύρεση της μεταβατικής κλειστότητας ενός γραφήματος. Επίσημως, αν $G = (V, E)$ είναι ένα γράφημα, τότε η **μεταβατική κλειστότητα** του G ορίζεται ως το γράφημα $G^* = (V, E^*)$, όπου $E^* = \{(v_i, v_j) \mid \text{υπάρχει μονοπάτι από το } v_i \text{ στο } v_j \text{ στον } G\}$. Υπολογίζουμε τη μεταβατική κλειστότητα ενός γραφήματος υπολογίζοντας τον πίνακα συνδεσιμότητας A^* . Ο **πίνακας συνδεσιμότητας** του G είναι ένας πίνακας $A^* = (a_{i,j}^*)$ τέτοιος ώστε $a_{i,j}^* = 1$ αν υπάρχει μονοπάτι από το v_i στο v_j ή διαφορετικά, $i = j$, και $a_{i,j}^* = \infty$.

Για τον υπολογισμό του A^* αναθέτουμε βάρος 1 σε κάθε ακμή του E και χρησιμοποιούμε οποιονδήποτε από τους αλγορίθμους συντομότερων μονοπατιών όλων των ζευγών σε αυτό το γράφημα με βάρη. Ο πίνακας A^* μπορεί να προκύψει από τον πίνακα D , όπου D είναι η λύση στο πρόβλημα των συντομότερων μονοπατιών όλων των ζευγών, ως εξής:

$$a_{i,j}^* = \begin{cases} \infty & \text{αν } d_{i,j} = \infty \\ 1 & \text{αν } d_{i,j} > 0 \text{ ή } i = j \end{cases}$$

Μια άλλη μέθοδος για τον υπολογισμό του A^* είναι να χρησιμοποιηθεί ο αλγόριθμος του Floyd στον πίνακα γειτνίασης του G , αντικαθιστώντας τις πράξεις $\min$ και $+$ στη γραμμή 7 του Αλγορίθμου 5 με λογικό **or** και λογικό **and** αντίστοιχα. Σε αυτή την περίπτωση, αρχικά ορίζουμε $a_{i,j} = 1$ αν $i = j$ ή διαφορετικά, $(v_i, v_j) \in E$, και $a_{i,j} = 0$. Ο πίνακας A^* προκύπτει θέτοντας $a_{i,j}^* = \infty$ αν $d_{i,j} = 0$ και $a_{i,j}^* = 1$ διαφορετικά. Η πολυπλοκότητα υπολογισμού της μεταβατικής κλειστότητας είναι $\Theta(n^3)$.

6 Συνεκτικές Συνιστώσες

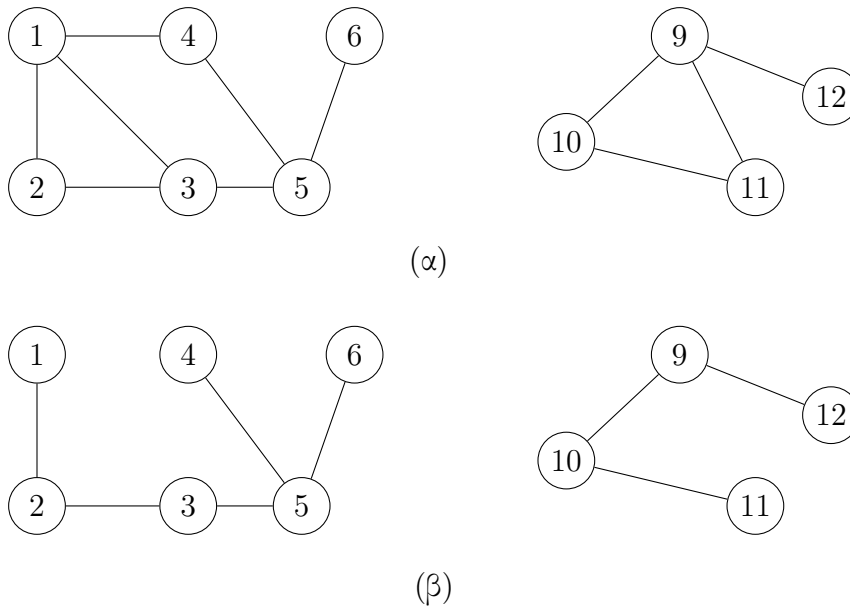
Οι **συνεκτικές συνιστώσες** ενός μη κατευθυνόμενου γραφήματος $G = (V, E)$ είναι τα μέγιστα ξένα μεταξύ τους σύνολα $C_1, C_2, \dots, C_k$, τέτοια ώστε $V = C_1 \cup C_2 \cup \dots \cup C_k$, και $u, v \in C_i$ αν και μόνο αν ο κόμβος u είναι προσπελάσιμος από τον v και ο κόμβος v είναι προσπελάσιμος από τον u . Οι συνεκτικές συνιστώσες ενός μη κατευθυνόμενου γραφήματος είναι οι κλάσεις ισοδυναμίας των κορυφών ως προς τη σχέση «είναι προσπελάσιμος από». Για παράδειγμα, το Σχήμα 16 δείχνει ένα γράφημα με τρεις συνεκτικές συνιστώσες.



Σχήμα 16: Ένα γράφημα με τρεις συνεκτικές συνιστώσες: $\{1, 2, 3, 4\}$, $\{5, 6, 7\}$, και $\{8, 9\}$.

6.1 Αλγόριθμος Βασισμένος στην Αναζήτηση Πρώτα Κατά Βάθος

Μπορούμε να βρούμε τις συνεκτικές συνιστώσες ενός γραφήματος εκτελώντας μια διάσχιση κατά βάθος (depth-first traversal) στο γράφημα. Το αποτέλεσμα αυτής της διάσχισης είναι ένα δάσος από δέντρα πρώτα κατά βάθος. Κάθε δέντρο του δάσους περιέχει κορυφές που ανήκουν σε διαφορετική συνεκτική συνιστώσα. Το Σχήμα 17 απεικονίζει αυτόν τον αλγόριθμο. Η ορθότητα του αλγορίθμου προκύπτει άμεσα από τον ορισμό του δέντρου επικάλυψης (δηλαδή, ένα δέντρο πρώτα κατά βάθος είναι επίσης ένα δέντρο επικάλυψης του γραφήματος που επάγεται από το σύνολο των κορυφών του δέντρου πρώτα κατά βάθος) και από το γεγονός ότι το G είναι μη κατευθυνόμενο. Ο χρόνος εκτέλεσης του αλγορίθμου είναι $\Theta(|E|)$, διότι ο αλγόριθμος διάσχισης κατά βάθος διατρέχει όλες τις ακμές του G .



Σχήμα 17: Το μέρος (β) είναι ένα δάσος πρώτα κατά βάθος που προκύπτει από διάσχιση πρώτα κατά βάθος του γραφήματος στο μέρος (α). Κάθε ένα από αυτά τα δέντρα είναι μια συνεκτική συνιστώσα του γραφήματος στο μέρος (α).

Παράλληλη Υλοποίηση

Ο αλγόριθμος των συνεκτικών συνιστώσων μπορεί να παραλληλοποιηθεί διαμερίζοντας τον πίνακα γειτνίασης του G σε p τμήματα και αναθέτοντας κάθε τμήμα σε έναν από τους p επεξεργαστές. Κάθε επεξεργαστής P_i έχει ένα υπογράφημα G_i του G , όπου $G_i = (V, E_i)$ και E_i είναι οι ακμές που αντιστοιχούν στο τμήμα του πίνακα γειτνίασης που έχει ανατεθεί στον συγκεκριμένο επεξεργαστή. Στο πρώτο βήμα αυτής της παράλληλης υλοποίησης, κάθε επεξεργαστής P_i υπολογίζει το δάσος επικάλυψης πρώτα κατά βάθος του υπογραφήματος G_i . Στο τέλος αυτού του βήματος, έχουν κατασκευαστεί p δάση επικάλυψης. Κατά το δεύτερο βήμα, τα δάση επικάλυψης συγχωνεύονται ανά ζεύγη έως ότου απομείνει ένα μόνο δάσος επικάλυψης. Το εναπομείνον δάσος επικάλυψης έχει την ιδιότητα ότι δύο κορυφές βρίσκονται στην ίδια συνεκτική συνιστώσα του G αν και μόνο αν ανήκουν στο ίδιο δέντρο. Το Σχήμα 18 απεικονίζει αυτόν τον αλγόριθμο.

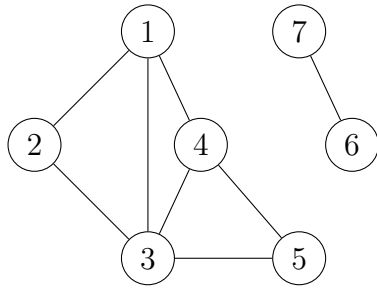
Για να συγχωνεύσουμε αποδοτικά ζεύγη δασών επικάλυψης, ο αλγόριθμος χρησιμοποιεί ξένα μεταξύ τους σύνολα ακμών. Ας υποθέσουμε ότι κάθε δέντρο στο δάσος επικάλυψης ενός υπογραφήματος του G αναπαρίσταται από ένα σύνολο. Τα σύνολα για διαφορετικά δέντρα είναι ανά ζεύγη ξένα μεταξύ τους. Οι ακόλουθες πράξεις ορίζονται πάνω στα ξένα σύνολα:

find(x) επιστρέφει έναν δείκτη στο αντιπροσωπευτικό στοιχείο του συνόλου που περιέχει το x . Κάθε σύνολο έχει το δικό του μοναδικό αντιπροσωπευτικό στοιχείο.

union(x, y) ενώνει τα σύνολα που περιέχουν τα στοιχεία x και y . Τα δύο σύνολα θεωρούνται ξένα μεταξύ τους πριν από την πράξη.

Τα δάση επικάλυψης συγχωνεύονται ως εξής. Έστω A και B τα δύο δάση επικάλυψης προς συγχώνευση. Το πολύ $n - 1$ ακμές (αφού τα A και B είναι δάση) του ενός συγχωνεύονται με τις ακμές του άλλου. Ας υποθέσουμε ότι θέλουμε να συγχωνεύσουμε το δάσος A στο δάσος B . Για κάθε ακμή (u, v) του A , εκτελείται μια πράξη **find** για κάθε κορυφή, προκειμένου να διαπιστωθεί αν οι δύο κορυφές ανήκουν ήδη στο ίδιο δέντρο του B . Αν όχι, τότε τα δύο δέντρα (σύνολα) του B που περιέχουν τις κορυφές u και v ενώνονται με μια πράξη **union**. Διαφορετικά, δεν απαιτείται καμία πράξη **union**. Επομένως, η συγχώνευση των A και B απαιτεί το πολύ $2(n - 1)$ πράξεις **find** και $(n - 1)$ πράξεις **union**. Μπορούμε να υλοποιήσουμε τη δομή δεδομένων ξένων συνόλων χρησιμοποιώντας δάση ξένων συνόλων με κατάταξη και συμπίεση διαδρομής. Με αυτή την υλοποίηση, το κόστος εκτέλεσης $2(n - 1)$ πράξεων **find** και $(n - 1)$ πράξεων **union** είναι $O(n)$. Μια λεπτομερής περιγραφή του δάσους ξένων συνόλων ξεφεύγει από το πεδίο αυτού του βιβλίου.

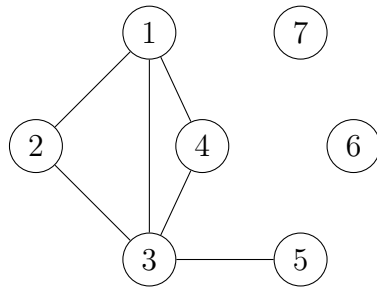
Αφού συζητήσαμε πώς να συγχωνεύουμε αποδοτικά δύο δάση επικάλυψης, εστιάζουμε τώρα στο πώς να διαμερίζουμε τον πίνακα γειτνίασης του G και να τον κατανέμουμε μεταξύ p επεξεργαστών. Η επόμενη ενότητα συζητά μια υλοποίηση που χρησιμοποιεί απεικόνιση με μπλοκ σε λωρίδες.



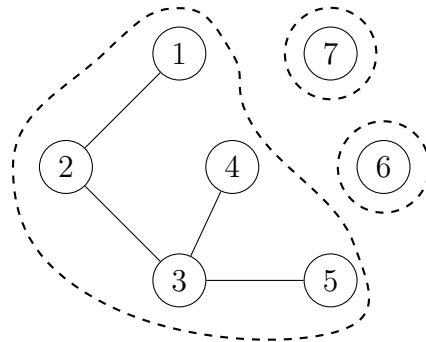
(α)

	1	2	3	4	5	6	7	
1	0	1	1	1	0	0	0	Επεξεργαστής 1
2	1	0	1	0	0	0	0	
3	1	1	0	1	1	0	0	
4	1	0	1	0	1	0	0	
5	0	0	1	1	0	0	0	Επεξεργαστής 2
6	0	0	0	0	0	0	1	
7	0	0	0	0	0	1	0	

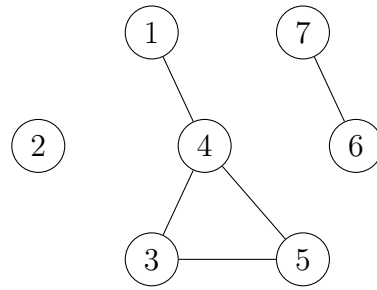
(β)



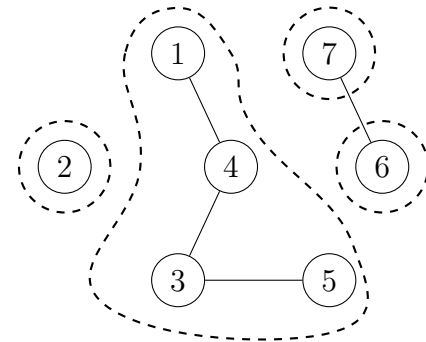
(γ)



(δ)



(ε)



(ζ)

Σχήμα 18: Υπολογισμός συνεκτικών συνιστωσών σε παραλληλία. Ο πίνακας γειτνίασης του γραφήματος G στο (α) διαμερίζεται σε δύο μέρη όπως φαίνεται στο (β). Στη συνέχεια, κάθε επεξεργαστής λαμβάνει ένα υπογράφημα του G όπως φαίνεται στα (γ) και (ε). Κάθε επεξεργαστής υπολογίζει στη συνέχεια το δάσος επικάλυψης του υπογραφήματος, όπως φαίνεται στα (δ) και (ζ). Τέλος, τα δύο δέντρα επικάλυψης συγχωνεύονται για να σχηματίσουν τη λύση.

Απεικόνιση με μπλοκ σε λωρίδες (Block-Striped Mapping)

Ο πίνακας γειτνίασης $n \times n$ διαμερίζεται σε p λωρίδες. Κάθε λωρίδα αποτελείται από n/p διαδοχικές γραμμές και ανατίθεται σε έναν από τους p επεξεργαστές. Για τον υπολογισμό των συνεκτικών συνιστωσών, κάθε επεξεργαστής υπολογίζει πρώτα ένα δάσος επικάλυψης για το γράφημα n κορυφών που αναπαρίσταται από τις n/p γραμμές του πίνακα γειτνίασης που του έχουν ανατεθεί.

Ας θεωρήσουμε έναν υπερκύβο p επεξεργαστών. Ο υπολογισμός του δάσους επικάλυψης βάσει του πίνακα γειτνίασης $(n/p) \times n$ που ανατίθεται σε κάθε επεξεργαστή απαιτεί χρόνο $\Theta(n^2/p)$. Το δεύτερο βήμα του αλγορίθμου, η ανά ζεύγη συγχώνευση δασών επικάλυψης, εκτελείται με την ενσωμάτωση ενός εικονικού δέντρου στον υπερκύβο. Υπάρχουν $\log p$ στάδια συγχώνευσης, και κάθε ένα απαιτεί χρόνο $\Theta(n)$. Επομένως, το κόστος λόγω συγχώνευσης είναι $\Theta(n \log p)$. Τέλος, κατά τη διάρκεια κάθε σταδίου συγχώνευσης, τα δάση επικάλυψης αποστέλλονται μεταξύ γειτονικών επεξεργαστών. Υπενθυμίζεται ότι μεταδίδονται $\Theta(n)$ ακμές του δάσους επικάλυψης. Επομένως, το κόστος επικοινωνίας είναι $\Theta(n \log p)$. Ο παράλληλος χρόνος εκτέλεσης του αλγορίθμου συνεκτικών συνιστωσών είναι:

$$T_P = \overbrace{\Theta\left(\frac{n^2}{p}\right)}^{\text{τοπικός υπολογισμός}} + \overbrace{\Theta(n \log p)}^{\text{συγχώνευση δασών}}$$

Εφόσον η σειριακή πολυπλοκότητα είναι $W = \Theta(n^2)$, η επιτάχυνση και η απόδοση έχουν ως εξής:

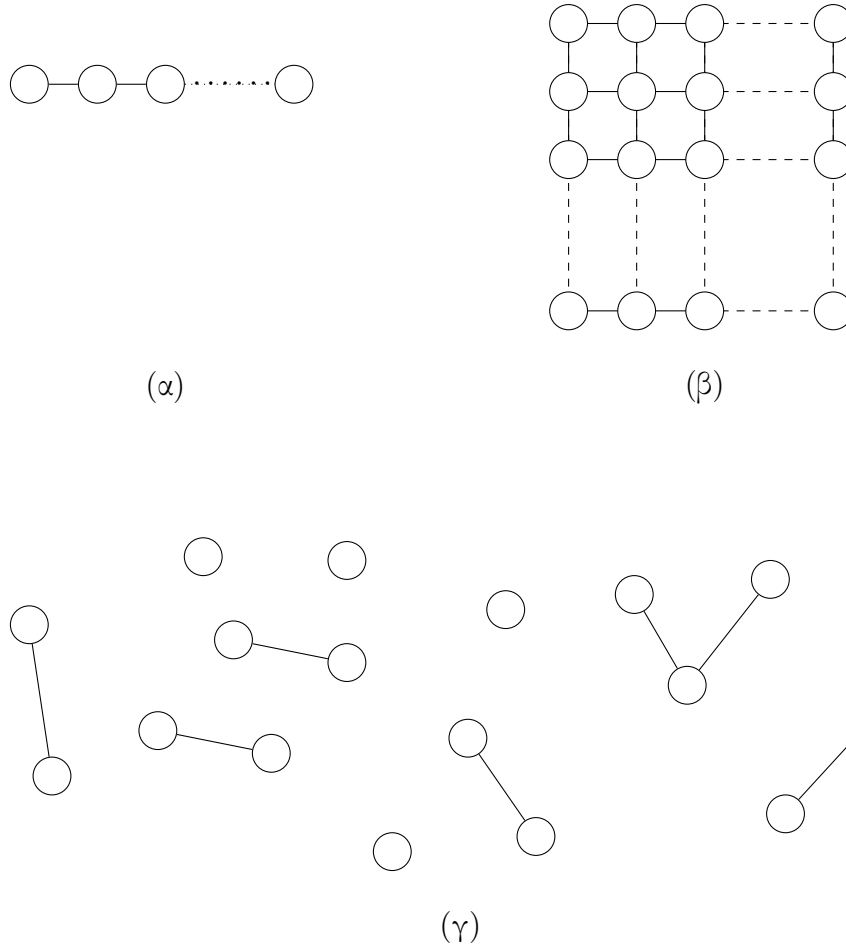
$$S = \frac{\Theta(n^2)}{\Theta(n^2/p) + \Theta(n \log p)}$$

$$E = \frac{1}{1 + \Theta((p \log p)/n)} \quad (11)$$

Από την Εξίσωση 11 βλέπουμε ότι για μια βέλτιστη ως προς κόστος υλοποίηση ισχύει $p = O(n/\log n)$. Επίσης από την Εξίσωση 11 εξάγουμε τη συνάρτηση ισοαποδοτικότητας, η οποία είναι $\Theta(p^2 \log^2 p)$. Αυτή είναι η συνάρτηση ισοαποδοτικότητας λόγω επικοινωνίας και λόγω των επιπλέον υπολογισμών που εκτελούνται στο στάδιο συγχώνευσης. Η συνάρτηση ισοαποδοτικότητας λόγω παραλληλισμού είναι $\Theta(p^2)$. Επομένως, η συνολική συνάρτηση ισοαποδοτικότητας είναι $\Theta(p^2 \log^2 p)$. Η απόδοση αυτής της παράλληλης υλοποίησης είναι παρόμοια με εκείνη του αλγορίθμου ελάχιστου δέντρου επικάλυψης του Prim και του αλγορίθμου συντομότερων μονοπατιών μονής πηγής του Dijkstra σε υπερκύβο. Η απόδοση αυτού του αλγορίθμου σε πλέγμα μπορεί να υπολογιστεί παρομοίως.

7 Αλγόριθμοι για Αραιά Γραφήματα

Οι παράλληλοι αλγόριθμοι των προηγούμενων ενοτήτων βασίζονται στους καλύτερα γνωστούς αλγόριθμους για προβλήματα πυκνών γραφημάτων. Ωστόσο, δεν έχουμε ακόμα ασχοληθεί με παράλληλους αλγόριθμους για αραιά γραφήματα. Υπενθυμίζουμε ότι ένα γράφημα $G = (V, E)$ είναι αραιό αν το $|E|$ είναι πολύ μικρότερο από το $|V|^2$. Το Σχήμα 19 δείχνει μερικά παραδείγματα αραιών γραφημάτων.



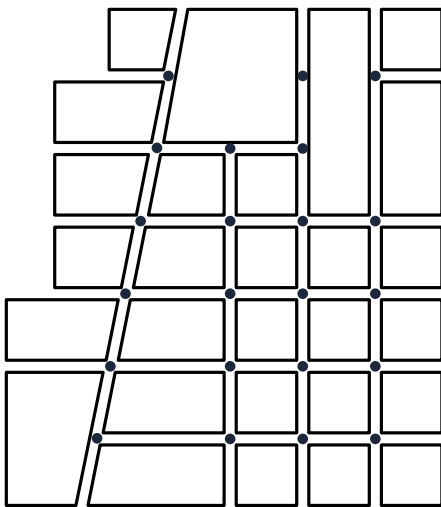
Σχήμα 19: Παραδείγματα αραιών γραφημάτων: (α) ένα γραμμικό γράφημα, στον οποίο κάθε κορυφή έχει δύο προσπίπτουσες ακμές, (β) ένα γράφημα πλέγματος, στον οποίο κάθε κορυφή έχει τέσσερις γειτονικές κορυφές και (γ) τυχαίο αραιό γράφημα.

Κάθε αλγόριθμος πυκνών γραφημάτων λειτουργεί σωστά και σε αραιά γραφήματα. Ωστόσο, αν ληφθεί υπόψη η αραιότητα του γραφήματος, είναι συνήθως δυνατό να επιτευχθεί σημαντικά καλύτερη απόδοση. Για παράδειγμα, ο χρόνος εκτέλεσης του αλγορίθμου ελάχιστου δέντρου επικάλυψης του Prim (Ενότητα 2) είναι $\Theta(n^2)$ ανεξάρτητα από τον αριθμό των ακμών του γραφήματος. Τροποποιώντας τον αλγόριθμο του Prim ώστε να χρησιμοποιεί λίστες γειτνίασης και δυαδικό σωρό, η πολυπλοκότητα του αλγορίθμου μειώνεται σε $\Theta(|E| \log n)$. Αυτός ο τροποποιημένος αλγόριθμος υπερτερεί του αρχικού εφόσον $|E| = O(n^2 / \log n)$. Ένα σημαντικό βήμα στην ανάπτυξη αλγορίθμων για αραιά γραφήματα είναι η χρήση λίστες γειτνίασης αντί πίνακα γειτνίασης. Αυτή η αλλαγή αναπαράστασης είναι κρίσιμη, καθώς η πολυπλοκότητα αλγορίθμων βασισμένων σε πίνακα γειτνίασης είναι συνήθως $\Omega(n^2)$, ανεξάρτητα από τον αριθμό των ακμών. Αντίθετα, η πολυπλοκότητα αλγορίθμων βασισμένων σε λίστα γειτνίασης είναι συνήθως $\Omega(n + |E|)$, η οποία εξαρτάται από την αραιότητα του γραφήματος.

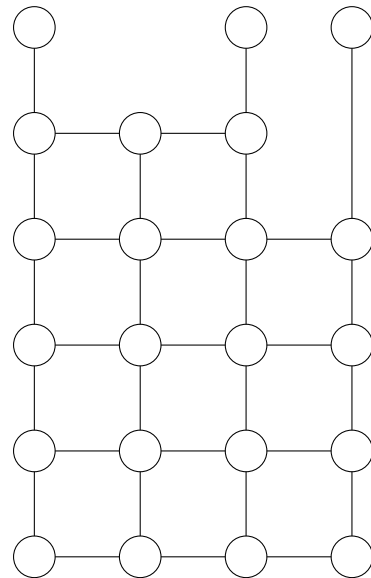
Στις παράλληλες υλοποιήσεις σειριακών αλγορίθμων για πυκνά γραφήματα, επιτύχαμε καλή απόδοση διαμερίζοντας τον πίνακα γειτνίασης ενός γραφήματος έτσι ώστε κάθε επεξεργαστής να εκτελεί περίπου ίσο φόρτο εργασίας και η επικοινωνία να είναι τοπική. Καταφέραμε να το επιτύχουμε αυτό σε μεγάλο βαθμό επειδή το γράφημα ήταν πυκνό. Για παράδειγμα, ας θεωρήσουμε τον αλγόριθμο συντομότερων μονοπατιών όλων των ζευγών του Floyd. Αναθέτοντας ισομεγέθη μπλοκ από τον πίνακα γειτνίασης σε όλους τους επεξερ-

γαστές, η εργασία κατανεμήθηκε ομοιόμορφα. Επιπλέον, εφόσον κάθε μπλοκ αποτελούνταν από διαδοχικές σειρές και στήλες, η επιβάρυνση επικοινωνίας ήταν περιορισμένη.

Ωστόσο, είναι δύσκολο να επιτευχθεί ομοιόμορφη κατανομή εργασίας και χαμηλή επιβάρυνση επικοινωνίας για αραιά γραφήματα. Θεωρήστε το πρόβλημα της κατάτμησης της λίστας γειτνίασης ενός γραφήματος. Μια πιθανή κατάτμηση αναθέτει έναν ίσο αριθμό κορυφών και των λιστών γειτνίασής τους σε κάθε επεξεργαστή. Ωστόσο, ο αριθμός των ακμών που προσπίπτουν σε μια δεδομένη κορυφή είναι ουσιαστικά τυχαίος. Επομένως, σε ορισμένους επεξεργαστές μπορεί να ανατεθεί ένας μεγάλος αριθμός ακμών, ενώ άλλοι να λάβουν πολύ λίγες, οδηγώντας σε σημαντική ανισορροπία φόρτου εργασίας μεταξύ των επεξεργαστών. Εναλλακτικά, μπορούμε να αναθέσουμε έναν ίσο αριθμό ακμών σε κάθε επεξεργαστή. Αυτό μπορεί να απαιτεί τον διαχωρισμό της λίστας γειτνίασης μιας κορυφής μεταξύ των επεξεργαστών. Ως αποτέλεσμα, ο χρόνος που δαπανάται για την επικοινωνία πληροφοριών μεταξύ επεξεργαστών που αποθηκεύουν ξεχωριστά τμήματα της λίστας γειτνίασης μπορεί να αυξηθεί δραματικά. Έτσι, είναι δύσκολο να παραχθούν αποτελεσματικές παράλληλες υλοποιήσεις για γενικά αραιά γραφήματα. Ωστόσο, μπορούμε συχνά να παραγάγουμε αποτελεσματικές παράλληλες υλοποιήσεις εάν το αραιό γράφημα έχει μια συγκεκριμένη δομή. Για παράδειγμα, θεωρήστε τον χάρτη δρόμων που φαίνεται στο Σχήμα 20. Το γράφημα που αντιστοιχεί στον χάρτη είναι αραιό: ο αριθμός των ακμών που προσπίπτουν σε οποιαδήποτε κορυφή είναι το πολύ τέσσερα. Αναφερόμαστε σε τέτοια γραφήματα ως **γραφήματα πλέγματος** (grid graphs). Η επόμενη ενότητα παρουσιάζει αποτελεσματικούς αλγορίθμους για τον υπολογισμό συντομότερων διαδρομών μονής πηγής για τέτοια γραφήματα.



(α)



(β)

Σχήμα 20: Ένας χάρτης οδών (α) μπορεί να αναπαρασταθεί ως γράφημα (β). Στο γράφημα του (β), κάθε διασταύρωση οδών είναι μια κορυφή και κάθε ακμή είναι ένα τμήμα οδού. Οι κορυφές του (β) είναι οι διασταυρώσεις του (α) που σημειώνονται με τελείες.

```

1: procedure JOHNSON_SINGLE_SOURCE_SP( $V, E, s$ )
2:    $Q := V$ 
3:   for all  $v \in Q$  do
4:      $l[v] := \infty$ 
5:   end for
6:    $l[s] := 0$ 
7:   while  $Q \neq \emptyset$  do
8:      $u := \text{EXTRACT\_MIN}(Q)$ 
9:     for all  $v \in \text{Adj}[u]$  do
10:      if  $v \in Q$  and  $l[u] + w(u, v) < l[v]$  then
11:         $l[v] := l[u] + w(u, v)$ 
12:      end if
13:    end for
14:   end while
15: end procedure

```

Αλγόριθμος 7: Σειριακός αλγόριθμος συντομότερων μονοπατιών μονής πηγής του Johnson.

7.1 Συντομότερα Μονοπάτια Μονής Πηγής

Είναι εύκολο να τροποποιήσουμε τον αλγόριθμο συντομότερων μονοπατιών μονής πηγής του Dijkstra έτσι ώστε να βρίσκει αποδοτικά τα συντομότερα μονοπάτια για αραιά γραφήματα. Ο τροποποιημένος αλγόριθμος είναι γνωστός ως αλγόριθμος του Johnson. Υπενθυμίζουμε ότι ο αλγόριθμος του Dijkstra εκτελεί τα εξής δύο βήματα σε κάθε επανάληψη. Πρώτον, εξάγει μια κορυφή $u \in (V - V_T)$ τέτοια ώστε $l[u] = \min\{l[v] \mid v \in (V - V_T)\}$ και την εισάγει στο σύνολο V_T . Δεύτερον, για κάθε κορυφή $v \in (V - V_T)$, υπολογίζει $l[v] = \min\{l[v], l[u] + w(u, v)\}$. Σημειώνουμε ότι κατά το δεύτερο βήμα, αρκεί να εξεταστούν μόνο οι κορυφές της λίστας γειτνίασης της κορυφής u . Εφόσον το γράφημα είναι αραιό, ο αριθμός των κορυφών που γειτνιάζουν με την u είναι σημαντικά μικρότερος από $\Theta(n)$. Επομένως, η χρήση της αναπαράστασης με λίστα γειτνίασης βελτιώνει την απόδοση.

Ο αλγόριθμος του Johnson χρησιμοποιεί μια ουρά προτεραιότητας Q για την αποθήκευση της τιμής $l[v]$ για κάθε κορυφή $v \in (V - V_T)$. Η ουρά προτεραιότητας κατασκευάζεται έτσι ώστε η κορυφή με τη μικρότερη τιμή στο l να βρίσκεται πάντα στην κορυφή της ουράς. Ένας συνηθισμένος τρόπος υλοποίησης ουράς προτεραιότητας είναι ένας μικρός δυαδικός σωρός. Ο μικρός δυαδικός σωρός επιτρέπει την ενημέρωση της τιμής $l[v]$ για κάθε κορυφή v σε χρόνο $O(\log n)$. Ο αλγόριθμος 7 παρουσιάζει τον αλγόριθμο του Johnson. Αρχικά, για κάθε κορυφή v εκτός της πηγής, εισάγεται $l[v] = \infty$ στην ουρά προτεραιότητας. Για την κορυφή πηγής s εισάγεται $l[s] = 0$. Σε κάθε βήμα του αλγορίθμου, η κορυφή $u \in (V - V_T)$ με τη μικρότερη τιμή στο l αφαιρείται από την ουρά προτεραιότητας. Στη συνέχεια διατρέχεται η λίστα γειτνίασης του u , και για κάθε ακμή (u, v) ενημερώνεται η απόσταση $l[v]$ προς την κορυφή v στον σωρό. Η ενημέρωση κορυφών στον σωρό κυριαρχεί στον συνολικό χρόνο εκτέλεσης του αλγορίθμου. Ο συνολικός αριθμός ενημερώσεων ισούται με τον αριθμό των ακμών. Επομένως, η συνολική πολυπλοκότητα του αλγορίθμου του Johnson είναι $\Theta(|E| \log n)$.

Παράλληλη Υλοποίηση

Μια αποδοτική παράλληλη υλοποίηση του αλγορίθμου του Johnson πρέπει να διατηρεί αποδοτικά την ουρά προτεραιότητας Q . Μια στρατηγική είναι να αναθέσουμε τη διαχείριση του Q σε έναν μόνο επεξεργαστή. Όλοι οι υπόλοιποι επεξεργαστές υπολογίζουν νέες τιμές του $l[v]$ για $v \in (V - V_T)$. Εφόσον σε κάθε επανάληψη ο αλγόριθμος ενημερώνει περίπου $|E|/|V|$ κορυφές, κατά μέσο όρο δεν μπορούν να απασχολούνται περισσότεροι από $|E|/|V|$ επεξεργαστές. Επιπλέον, κάθε επεξεργαστής αποστέλλει την τιμή του l που υπολογίζει στον επεξεργαστή που διαχειρίζεται την ουρά προτεραιότητας. Αυτός ο επεξεργαστής ενημερώνει την ουρά προτεραιότητας μόλις λάβει νέες τιμές του $l[v]$ για $v \in (V - V_T)$. Εφόσον κάθε πρόσβαση στην ουρά απαιτεί χρόνο $O(\log n)$, ο παράλληλος χρόνος εκτέλεσης είναι $O(|E| \log n)$. Σημειώνουμε ότι αυτός είναι ίδιος με τον σειριακό χρόνο εκτέλεσης. Το κύριο μειονέκτημα αυτού του σχήματος είναι η κεντροποιημένη ουρά προτεραιότητας. Ένας τρόπος να ανακουφιστεί αυτό το μειονέκτημα είναι να κατανεμηθεί η ουρά προτεραιότητας. Στο υπόλοιπο αυτής της ενότητας περιγράφουμε μια προσέγγιση για την επίτευξη αυτής της κατανομής.

Έστω p ο αριθμός των επεξεργαστών και $G = (V, E)$ ένα αραιό γράφημα. Διαμερίζουμε το σύνολο κορυφών V σε p ξένα σύνολα $V_1, V_2, \dots, V_p$, και αναθέτουμε κάθε σύνολο κορυφών μαζί με τις αντίστοιχες λίστες γειτνιάσής του σε έναν από τους p επεξεργαστές. Κάθε επεξεργαστής διατηρεί μια ουρά προτεραιότητας για τις κορυφές που του έχουν ανατεθεί και υπολογίζει τα συντομότερα μονοπάτια από την πηγή προς αυτές τις κορυφές. Έτσι, η ουρά προτεραιότητας Q διαμερίζεται σε p ξένες ουρές προτεραιότητας $Q_1, Q_2, \dots, Q_p$, καθεμία ανατεθειμένη σε έναν ξεχωριστό επεξεργαστή. Επιπλέον, κάθε επεξεργαστής P_i διατηρεί έναν πίνακα sp τέτοιον ώστε το $sp[v]$ να αποθηκεύει το κόστος του συντομότερου μονοπατιού από την κορυφή πηγής προς το v για κάθε κορυφή $v \in V_i$. Το κόστος $sp[v]$ ενημερώνεται στην τιμή $l[v]$ κάθε φορά που η κορυφή v εξάγεται από την ουρά προτεραιότητας. Αρχικά, $sp[v] = \infty$ για κάθε κορυφή v εκτός της πηγής, και εισάγουμε $l[s]$ στην κατάλληλη ουρά προτεραιότητας για την κορυφή πηγής s . Κάθε επεξεργαστής εκτελεί τον αλγόριθμο του Johnson στην τοπική του ουρά προτεραιότητας. Στο τέλος του αλγορίθμου, το $sp[v]$ αποθηκεύει το μήκος του συντομότερου μονοπατιού από την πηγή προς την κορυφή v .

Όταν ο επεξεργαστής P_i εξάγει την κορυφή $u \in V_i$ με τη μικρότερη τιμή $l[u]$ από το Q_i , οι τιμές l των κορυφών που έχουν ανατεθεί σε άλλους επεξεργαστές πέρα από τον P_i , ενδέχεται να χρειαστούν ενημέρωση. Ο επεξεργαστής P_i αποστέλλει μήνυμα στους επεξεργαστές με σχετικές κορυφές, ενημερώνοντάς τους για τις νέες τιμές. Μόλις λάβουν αυτές τις τιμές, οι επεξεργαστές ενημερώνουν τις τιμές του l . Για παράδειγμα, ας υποθέσουμε ότι υπάρχει μια ακμή (u, v) τέτοια ώστε $u \in V_i$ και $v \in V_j$, και ότι ο επεξεργαστής P_i μόλις εξήγαγε την κορυφή u από την ουρά προτεραιότητάς του. Τότε ο επεξεργαστής P_i αποστέλλει μήνυμα στον P_j που περιέχει την πιθανή νέα τιμή του $l[v]$, η οποία είναι $l[u] + w(u, v)$. Ο επεξεργαστής P_j , μόλις λάβει αυτό το μήνυμα, θέτει την τιμή του $l[v]$ που είναι αποθηκευμένη στην ουρά προτεραιότητάς του ίση με $\min\{l[v], l[u] + w(u, v)\}$.

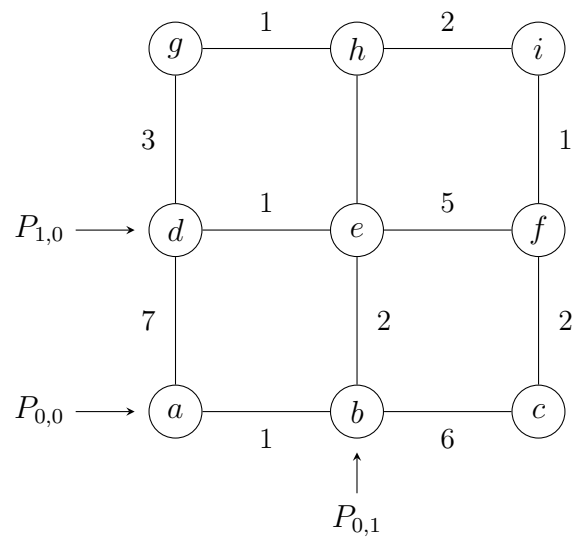
Εφόσον και οι δύο επεξεργαστές P_i και P_j εκτελούν τον αλγόριθμο του Johnson, είναι πιθανό ο επεξεργαστής P_j να έχει ήδη εξάγει την κορυφή v από την ουρά προτεραιότητάς του. Αυτό σημαίνει ότι ο επεξεργαστής P_j ενδέχεται να έχει ήδη υπολογίσει το συντομότερο μονοπάτι $sp[v]$ από την πηγή προς την κορυφή v . Τότε υπάρχουν δύο πιθανές περιπτώσεις: είτε $sp[v] \leq l[u] + w(u, v)$, είτε $sp[v] > l[u] + w(u, v)$. Η πρώτη περίπτωση σημαίνει ότι υπάρχει ένα μακρύτερο μονοπάτι προς την κορυφή v που διέρχεται από την κορυφή u , ενώ η δεύτερη περίπτωση σημαίνει ότι υπάρχει ένα συντομότερο μονοπάτι προς την κορυφή v που διέρχεται από την κορυφή u . Στην πρώτη περίπτωση, ο επεξεργαστής P_j δεν χρειάζεται να κάνει τίποτα, καθώς το συντομότερο μονοπάτι προς το v δεν αλλάζει. Στη δεύτερη περιπτώ-

ση, ο επεξεργαστής P_j πρέπει να ενημερώσει το κόστος του συντομότερου μονοπατιού προς την κορυφή v . Αυτό γίνεται εισάγοντας εκ νέου την κορυφή v στην ουρά προτεραιότητας με $l[v] = l[u] + w(u, v)$ αγνοώντας την τιμή του $sp[v]$. Εφόσον μια κορυφή v μπορεί να επανεισαχθεί στην ουρά προτεραιότητας, ο αλγόριθμος τερματίζει μόνο όταν όλες οι ουρές αδειάσουν.

Είναι πιθανό, ενώ ο επεξεργαστής P_i εξάγει μια κορυφή v από το Q_i , ένας άλλος επεξεργαστής P_j να έχει ήδη εξαγάγει μια κορυφή u από το Q_j τέτοια ώστε $l[v] < l[u]$. Ως αποτέλεσμα, αν υπάρχει ένα μονοπάτι από την κορυφή v προς την κορυφή u , το κόστος του συντομότερου μονοπατιού προς την κορυφή u ενδέχεται να μειωθεί μετά την εξαγωγή της κορυφής v . Αν υπήρχε μόνο μία ουρά προτεραιότητας, η κορυφή v θα είχε εξαχθεί νωρίτερα από την κορυφή u . Επομένως, ο κατανεμημένος αλγόριθμος δεν εξάγει κορυφές με μη φθίνουσα σειρά τιμών l . Ως αποτέλεσμα, εκτελεί υπολογισμούς που πρέπει να επαναληφθούν. Αυτοί οι ενδιάμεσοι υπολογισμοί αποτελούν επιπλέον εργασία που εκτελεί η παράλληλη υλοποίηση σε σχέση με τον σειριακό αλγόριθμο. Ανάλογα με το γράφημα, το ποσό της επιπλέον εργασίας μπορεί να είναι σημαντικό.

Παράδειγμα 1. Προβλήματα με την Κατανεμημένη Διαχείριση της Ουράς Προτεραιότητας *Ας* θεωρήσουμε το γράφημα πλέγματος που παρουσιάζεται στο Σχήμα 21. Κάθε κορυφή αυτού του γραφήματος ανατίθεται σε έναν ξεχωριστό επεξεργαστή, και η κορυφή a είναι η πηγή για τα συντομότερα μονοπάτια. Ο επεξεργαστής $P_{0,0}$ εξάγει την κορυφή a από την τοπική του ουρά και αποστέλλει τις τιμές $l[a] + w(a, b)$ και $l[a] + w(a, d)$ στους επεξεργαστές $P_{0,1}$ και $P_{1,0}$ αντίστοιχα. Αυτοί οι επεξεργαστές εισάγουν τις τιμές στις ουρές προτεραιότητάς τους και στη συνέχεια εκτελούν υπολογισμούς. *Ας* εξετάσουμε τον επεξεργαστή $P_{1,0}$. Εφόσον το $l[d]$ είναι η μόνη τιμή αποθηκευμένη στην ουρά προτεραιότητάς του, ο επεξεργαστής εξάγει την κορυφή d από αυτήν, υποθέτοντας ότι το μονοπάτι $\langle a, d \rangle$ είναι το συντομότερο μονοπάτι από την κορυφή a προς την κορυφή d . Όμως το μονοπάτι $\langle a, d \rangle$ δεν είναι το συντομότερο μονοπάτι από την κορυφή a προς την κορυφή d . Το συντομότερο μονοπάτι είναι το $\langle a, b, e, d \rangle$. Επομένως, σε μεταγενέστερο χρόνο, ο επεξεργαστής $P_{0,0}$ θα λάβει μια μικρότερη τιμή για το $l[d]$ που αντιστοιχεί στο πραγματικό συντομότερο μονοπάτι, και θα πρέπει να επαναλάβει ορισμένους από τους υπολογισμούς που έχει ήδη εκτελέσει. Επομένως, η κατανεμημένη διαχείριση της ουράς προτεραιότητας μπορεί να οδηγήσει σε επιπλέον υπολογισμούς.

Αρχικά, μόνο η ουρά προτεραιότητας του επεξεργαστή που διαθέτει την κορυφή πηγής είναι μη κενή. Στη συνέχεια, οι ουρές προτεραιότητας των υπόλοιπων επεξεργαστών αρχίζουν να πληρούνται καθώς δημιουργούνται μηνύματα με νέες τιμές l και αποστέλλονται στους γειτονικούς επεξεργαστές. Όταν οι επεξεργαστές λαμβάνουν νέες τιμές l , τις εισάγουν στις ουρές προτεραιότητάς τους και εκτελούν υπολογισμούς. *Ας* εξετάσουμε το πρόβλημα υπολογισμού των συντομότερων μονοπατιών μόνης πηγής σε ένα γράφημα πλέγματος όπου η πηγή



Σχήμα 21: Ένα γράφημα πλέγματος.

Βιβλιογραφία

- [1] Vipin Kumar et al. *Introduction to Parallel Computing: Design and Analysis of Algorithms*. Addison-Wesley, 2003.
- [2] Thomas H. Cormen et al. *Introduction to Algorithms*. MIT Press, 2009.
- [3] Mengqing He. «Parallelizing Dijkstra's Algorithm». Master's thesis. St. Cloud State University, Dec. 2020. URL: https://repository.stcloudstate.edu/csit_etds/35.